



CMP6200/DIG6200

Individual Undergraduate Project (FYP)

Final Report (Final Draft)

CropSense AI: An Explainable, Bilingual Deep-Learning System for Crop Leaf Disease Detection for Nepali Farmers



Student Name: Bibek Shah

Student ID: [Your ID Number]

Course: [Your Course Title]

Supervisor: [Your Supervisor's Name]

Date: [Date of Submission]

Abstract

Plant diseases destroy a substantial share of the world's staple crops, and smallholder farmers in Nepal often lack timely access to expert diagnosis in a language they read comfortably. Deep-learning classifiers report near-perfect accuracy on laboratory leaf images, yet they rarely reach farmers as usable tools, seldom explain their predictions, and are known to fail on real field photographs and on crops they were never trained on.

This project designed, built, deployed and critically evaluated **CropSense AI**, an end-to-end decision-support system that identifies 51 disease and healthy conditions across 10 crops from a single leaf photograph. The artefact comprises an EfficientNetB2 transfer-learning model (8.5 million parameters) trained in two phases on 38,348 laboratory images and 1,593 cleaned field images; a FastAPI inference service with a leaf pre-check, leaf cropping, a five-rule “unknown” gate combining softmax uncertainty with cosine distance to class centroids, and Grad-CAM visual explanations; a knowledge base of symptoms, causes and 122 structured treatment entries in English and Nepali; user accounts with bcrypt-hashed passwords, signed session tokens and a prediction history stored in a hosted PostgreSQL database, so that every check follows the user between phone and browser; a React web application; and an Expo/React Native Android application. The system is publicly deployed on Vercel, a Hugging Face Space and a Neon serverless database.

Software validation passed all 28 automated frontend tests and all 17 authentication and account-history API tests; a suite of API edge-case tests passed 8 of 9 cases and exposed one input-size defect. Because the training notebook's held-out metrics were not preserved, the deployed model was evaluated independently on 226 real-world photographs from the PlantDoc benchmark after removing near-duplicates of the training data. On supported classes the model achieved 48.0% top-1 and 81.8% top-5 accuracy (70.9% at crop level); predictions made with at least 80% confidence were correct 81.8% of the time. The open-set gate rejected only 28.4% of untrained-crop leaves, and centroid similarity did not separate known from unknown crops on this benchmark.

The project contributes a working, explainable and bilingual tool for Nepali users, together with an honest, reproducible field evaluation showing that the laboratory-to-field gap and open-set rejection remain the central unsolved problems. Priorities for future work are expert-verified Nepali field data, calibrated selective prediction and on-device offline inference.

Keywords: plant disease detection; transfer learning; EfficientNet; Grad-CAM; open-set recognition; domain shift; bilingual decision support; Nepal

Table of Contents

1.0 Introduction and Context.....	1
1.1 Problem Statement.....	1
1.2 Structure of the Report.....	2
2.0 Review of Existing Knowledge.....	3
2.0.1 Image-based plant disease classification.....	3
2.0.2 Transfer learning and efficient architectures.....	3
2.0.3 The laboratory-to-field domain gap.....	4
2.0.4 Uncertainty and open-set recognition.....	4
2.0.5 Explainability.....	4
2.0.6 Deployed tools for farmers.....	5
2.1 Critical Analysis and Gap Identification.....	5
2.2 Project Justification.....	6
3.0 Project Aims, Objectives, and Scope.....	7
3.1 Project Aim.....	7
3.2 Project Objectives.....	7
3.3 Project Scope.....	8
4.0 Project Design.....	9
4.1 Design and Methods.....	9
4.2 Justification of Methods.....	10
5. Artefact Design.....	13
5.1 Design Specifications.....	13
5.2 Artefact's Structure / Architecture / Framework.....	15
5.2.1 Model architecture.....	16
5.2.2 Inference pipeline and decision logic.....	16
5.2.3 API and response design.....	18
5.2.4 Knowledge base and interface design.....	18
5.2.5 Data flow and data design.....	19
5.2.6 Authentication and session design.....	21
6. Implementation.....	24
6.1 Construction of the Artefact.....	24
6.1.1 Data acquisition, cleaning and preparation.....	24
6.1.2 Model training.....	26
6.1.3 Open-set calibration.....	28
6.1.4 Inference API.....	28
6.1.5 Web application.....	29
6.1.6 Mobile application.....	34
6.1.7 Deployment.....	35
6.2 Validation and Testing.....	37
6.2.1 Automated software tests.....	37
6.2.2 Internal training evidence.....	38

6.2.3 API robustness and functional tests.....	39
6.2.4 Independent field evaluation on PlantDoc.....	40
6.2.5 Performance.....	45
7. Critical Evaluation.....	46
7.1 Evaluation Against Objectives and Existing Solutions.....	46
7.2 Impact.....	48
8. Conclusion.....	49
8.1 Limitations.....	49
8.2 Future Work.....	50
References.....	52
Appendices.....	56
A. Project Timeline.....	56
B. Feasibility, Risks, and Ethical Aspects.....	56
B.1. Revised Feasibility.....	56
B.2. Updated Risk Analysis and Mitigation.....	57
B.3 Revised Ethical Considerations.....	57
C. Additional Screenshots.....	59
D. Example API Response (abridged).....	61
E. Full Per-Class PlantDoc Results.....	62
F. Repository Structure and Running the Artefact.....	62

List of Figures

Figure 1: Iterative design-science development cycle used in the project.....	9
Figure 2: Use case diagram of CropSense AI (all diagnosis and history use cases require a signed-in session).....	13
Figure 3: System architecture of the deployed CropSense AI artefact, including the accounts database.....	15
Figure 4: Classifier architecture (EfficientNetB2 backbone with a regularised dense head).....	16
Figure 5: Request flow and decision logic of the /predict endpoint.....	17
Figure 6: Activity diagram of the user journey on the web and Android clients.....	19
Figure 7: Context (level 0) data-flow diagram of CropSense AI.....	20
Figure 8: Level 1 data-flow diagram: processes, data stores and flows.....	20
Figure 9: Entity-relationship diagram of the accounts database.....	21
Figure 10: Sequence diagram: sign-in, diagnosis with history saving, and history retrieval.....	22
Figure 11: Images per class in the laboratory and field datasets (logarithmic scale).....	24
Figure 12: Images and classes per crop.....	25
Figure 13: Data preparation and training pipeline.....	26
Figure 14: Learning-rate schedule across the two training phases (upper bound on epochs).....	27
Figure 15: Home page with a Nepali farming photograph (Payne, 2018; CC BY-SA 4.0) and a single primary action.....	30
Figure 16: Diagnose page: crop selection, uploaded photo preview and photo-quality tips.....	30
Figure 17: Result page for a PlantDoc apple leaf: possible match, confidence band and Grad-CAM comparison.....	31
Figure 18: Result page (continued): symptoms, cause, management, prevention and structured treatment options with hazard bands and disclaimer.....	32
Figure 19: Account prediction history with thumbnails, confidence status, search, filters and deletion.....	33
Figure 20: Sign-in and create-account screens, shown without header or footer until a session exists.....	33
Figure 21: Account menu in the header: username, email, profile link and sign-out.....	34
Figure 22: Profile page: number of saved checks, member-since date, account details and sign-out.....	34
Figure 23: Deployment pipeline for the web client, API and Android application.....	35
Figure 24: Training and validation accuracy and loss over 19 epochs for the 31 August 2026 laboratory run.....	38
Figure 25: Laboratory test-set confusion matrix from the 31 August 2026 run (51 classes, before the Unknown class was added).....	39
Figure 26: Per-class top-1 and top-5 accuracy on PlantDoc field photographs (supported classes).....	42
Figure 27: System-level outcomes on PlantDoc for supported and untrained crops.....	43
Figure 28: Accuracy versus coverage when answering only the most confident predictions (selective prediction).....	43
Figure 29: Distribution of the open-set score for supported and untrained crops on PlantDoc, with the calibrated threshold.....	44

Figure 30: Grad-CAM explanations for PlantDoc photographs: three correct predictions and one confident misclassification.....	44
Figure 31: End-to-end /predict latency: distribution during the local evaluation and mean sequential latency locally and on the live Space.....	45
Figure 32: Project Gantt chart with milestones.....	56
Figure 33: Result page in Nepali: every content field, including the confidence band and explanation text, is translated.....	59
Figure 34: Responsive web interface at phone width (390 px): result and treatment sections.....	60
Figure 35: Disease library listing all supported conditions.....	61
Figure 36: Android application screenshots (to be inserted from a physical device).....	61

List of Tables

Table 1: Comparison of representative plant disease detection studies and systems.....	5
Table 2: Project scope: inclusions and exclusions.....	8
Table 3: Methods mapped to project objectives.....	9
Table 4: Principal tools and technologies.....	10
Table 5: Comparison of candidate CNN backbones (ImageNet figures from Keras (2026)).....	11
Table 6: Functional requirements.....	13
Table 7: Non-functional requirements and constraints.....	14
Table 8: Architectural components and responsibilities.....	15
Table 9: Rules of the unknown gate implemented in the backend.....	17
Table 10: Public API endpoints.....	18
Table 11: Structure of the /predict response.....	18
Table 12: Database tables, content and access rules.....	21
Table 13: Security controls for accounts and history.....	22
Table 14: Datasets used in the project.....	26
Table 15: Field-realism augmentation applied during training.....	27
Table 16: Training hyperparameters.....	27
Table 17: Model iterations retained in the repository.....	28
Table 18: Deployed services.....	35
Table 19: Key construction challenges and decisions.....	36
Table 20: Validation and testing strategy.....	37
Table 21: Automated frontend test suite.....	37
Table 22: API black-box and edge-case test results.....	39
Table 23: Authentication and account-history API test results.....	40
Table 24: Mapping of the PlantDoc test split to evaluation groups.....	41
Table 25: Independent field evaluation results for supported classes.....	41
Table 26: Assessment of the project against its SMART objectives.....	46
Table 27: Comparison of CropSense AI with similar systems.....	47
Table 28: Limitations, their impact and how they were addressed.....	49
Table 29: Risk assessment and mitigation.....	57
Table 30: Per-folder PlantDoc results for the deployed model.....	62

1.0 Introduction and Context

Agriculture remains the economic foundation of Nepal. The sector contributes roughly a quarter of national gross domestic product and employs the majority of the working population, most of whom farm small, rain-fed plots in the hills and the Terai (World Bank, 2024; MoALD, 2023). For these households a single disease outbreak can erase a season's income. Globally, pathogens and pests are estimated to reduce harvests of the major staple crops by between 17% and 30% each year, with the heaviest losses concentrated in food-deficit regions with fast-growing populations (Savary *et al.*, 2019). Plant disease is therefore not only an agronomic problem but a threat to food security and rural livelihoods (Strange and Scott, 2005).

Effective control depends on early and correct diagnosis. Many diseases look alike at first sight: early blight and Septoria leaf spot on tomato, or common rust and northern leaf blight on maize, present similar lesions to an untrained eye, yet they are managed with different products and timings. When diagnosis is delayed or wrong, farmers either lose the crop or apply the wrong pesticide, which wastes money, harms beneficial organisms and leaves residues on food. In practice many Nepali farmers rely on neighbours or agro-input retailers (agrovets) for advice, and written guidance is often available only in English or in technical language.

Over the last decade, computer vision based on deep convolutional neural networks (CNNs) has transformed image-based plant disease recognition (Kamilaris and Prenafeta-Boldú, 2018; Liu and Wang, 2021). Seminal work showed that a CNN could distinguish 38 crop–disease classes from leaf photographs with over 99% accuracy (Mohanty, Hughes and Salathé, 2016), and the near-universal availability of smartphone cameras makes it plausible to put such a model in a farmer's pocket (Ramcharan *et al.*, 2019). At the same time, the field has become aware that these headline accuracies are measured on clean laboratory images and fall sharply on real field photographs, that models confidently mislabel crops they have never seen, and that opaque predictions are difficult for users to trust (Barbedo, 2018; Hendrycks and Gimpel, 2017; Adadi and Berrada, 2018). This project sits at the intersection of these themes: it applies modern transfer learning to crop disease detection, but treats field realism, uncertainty, explainability and language accessibility as first-class design requirements rather than afterthoughts.

The resulting artefact, **CropSense AI**, is a complete, publicly deployed system: a user photographs a leaf on the web or on an Android phone and, within seconds, receives a possible disease match, a confidence band, a heatmap showing where the model looked, and practical guidance on symptoms, causes, prevention and treatment, all available in English and Nepali. Every check is saved to the user's account, so the history is available on any phone or browser they sign in on.

1.1 Problem Statement

Smallholder farmers in Nepal lack an accessible, trustworthy and Nepali-language means of identifying crop leaf diseases early; existing deep-learning research

reports laboratory accuracies that do not transfer to real field photographs, rarely signals when it does not know, and seldom reaches farmers as a usable, explainable tool, leading to delayed or incorrect treatment and avoidable crop loss.

1.2 Structure of the Report

Section 2 critically reviews existing knowledge and identifies the research gap. Section 3 sets out the aim, SMART objectives and scope. Section 4 describes and justifies the design-science methodology. Section 5 specifies the artefact and its architecture, and Section 6 documents its construction, validation and testing, including an independent field evaluation. Section 7 critically evaluates the artefact against its objectives and against comparable systems and discusses its impact. Section 8 concludes with limitations and future work. Appendices contain the project timeline, the feasibility, risk and ethics analysis, and supporting technical material.

2.0 Review of Existing Knowledge

This review is organised around six themes that together determine whether an image-based disease detector is useful in practice: the evolution of CNN-based classification, transfer learning and efficient architectures, the laboratory-to-field domain gap, uncertainty and open-set recognition, explainability, and deployed tools for farmers.

2.0.1 Image-based plant disease classification

Early automated approaches relied on hand-engineered colour, texture and shape features fed to classifiers such as support vector machines; they worked on narrow problems but required expert feature design for every crop. The release of PlantVillage, an open repository of more than 50,000 expertly labelled leaf images photographed on uniform backgrounds (Hughes and Salathé, 2015), enabled end-to-end deep learning. Mohanty, Hughes and Salathé (2016) trained AlexNet and GoogLeNet on 54,306 PlantVillage images covering 14 crops and 26 diseases and reached 99.35% accuracy on a held-out split. Ferentinos (2018) extended this to 87,848 images of 25 plants in 58 classes, reporting 99.53% with a VGG architecture, and Too *et al.* (2019) compared fine-tuned VGG16, Inception-v4, ResNet and DenseNet, finding DenseNet-121 best at 99.75%. Surveys consistently conclude that CNNs outperform classical pipelines when sufficient labelled data are available (Kamilaris and Prenafeta-Boldú, 2018; Liu and Wang, 2021).

A critical reading of this literature shows that accuracy on PlantVillage-style data is close to saturated, so further architectural comparisons on the same data add little. The more important questions concern what these numbers mean outside the laboratory.

2.0.2 Transfer learning and efficient architectures

Because labelled agricultural images are expensive to collect, almost all successful systems use transfer learning: a network pre-trained on ImageNet (Deng *et al.*, 2009) is reused as a feature extractor and fine-tuned on the target task (Zhuang *et al.*, 2021). Fine-tuning typically outperforms training from scratch, particularly on small or imbalanced datasets (Too *et al.*, 2019). Architecture choice then becomes a trade-off between accuracy and cost. ResNet (He *et al.*, 2016) introduced residual connections that make deep networks trainable; MobileNetV2 (Sandler *et al.*, 2018) uses inverted residual blocks for mobile efficiency; and EfficientNet (Tan and Le, 2019) scales depth, width and resolution together with a compound coefficient, achieving higher ImageNet accuracy than much larger models with an order of magnitude fewer parameters. For a system that must be trained on free cloud GPUs and served on CPU-only free hosting, this parameter efficiency is a decisive practical property rather than a detail.

Training practice also matters. Label smoothing reduces over-confidence and improves calibration of the softmax output (Szegedy *et al.*, 2016; Müller, Kornblith and Hinton, 2019), which is directly relevant when confidence is shown to users. Class imbalance, common in disease datasets where some conditions are rare, biases CNNs towards majority classes; oversampling, undersampling and class weighting mitigate this, and macro-averaged metrics expose it (Buda, Maki and Mazurowski, 2018; Johnson and

Khoshgoftaar, 2019; Sokolova and Lapalme, 2009). Data augmentation remains the standard defence against overfitting on limited images (Shorten and Khoshgoftaar, 2019).

2.0.3 The laboratory-to-field domain gap

The most consistent critical finding in the literature is that models trained on laboratory images degrade severely on field photographs. Mohanty, Hughes and Salathé (2016) themselves reported that accuracy fell to 31.4% when their model was tested on images collected under different conditions. Ferentinos (2018) observed that models trained only on laboratory images performed poorly on field images, and that including field images in training was necessary. Barbedo (2018) showed that dataset size and, above all, variety of capture conditions govern real-world effectiveness, and argued that most published datasets under-represent the variability of practical use. PlantDoc, a dataset of 2,598 web-sourced field images of 13 crops, was created precisely because PlantVillage-trained models failed on real scenes (Singh *et al.*, 2020). Picon *et al.* (2019) similarly found that models must be trained on images captured with mobile devices in the field to be reliable. More broadly, distribution shift between development and deployment data is recognised as a central unsolved problem across machine learning (Koh *et al.*, 2021).

Two further issues compound the gap. Field images typically contain multiple leaves, soil, hands and sky, so lesion-focused approaches (Barbedo, 2019) or explicit leaf localisation are needed. And web-scraped field datasets contain label noise; even widely used benchmarks have measurable error rates in their test sets (Northcutt, Athalye and Mueller, 2021), which limits how precisely any field accuracy can be interpreted.

2.0.4 Uncertainty and open-set recognition

A standard classifier is closed-set: its softmax output must assign every input to one of the training classes, so a leaf from an untrained crop is forced onto the most similar known class, often with high confidence. Hendrycks and Gimpel (2017) established the maximum softmax probability as a baseline for detecting misclassified and out-of-distribution (OOD) inputs, but modern networks are frequently over-confident (Guo *et al.*, 2017), limiting this baseline. Feature-space methods, which measure the distance of an input's embedding from class-conditional statistics, often detect OOD inputs more reliably (Lee *et al.*, 2018), and open-set recognition has become an active research area (Geng, Huang and Chen, 2021). A related idea is selective classification: a system may abstain on low-confidence inputs to trade coverage for accuracy (Geifman and El-Yaniv, 2017). In agricultural decision support, where a wrong confident answer can lead to the purchase of the wrong pesticide, an explicit “not sure” is often the safer output. Despite this, very few published plant disease systems evaluate rejection of untrained crops at all.

2.0.5 Explainability

Explainable AI (XAI) aims to make model decisions interpretable to humans, which is important for trust and for detecting spurious behaviour (Adadi and Berrada, 2018). Gradient-weighted Class Activation Mapping (Grad-CAM) produces a coarse heatmap of

the image regions that most influenced a CNN's decision by weighting the final convolutional feature maps with the gradients of the class score (Selvaraju *et al.*, 2017). It requires no architectural change and is widely used to show that plant disease models attend to lesions. However, saliency methods must be interpreted carefully: Adebayo *et al.* (2018) demonstrated that some saliency maps remain visually plausible even when model parameters are randomised, so a convincing heatmap does not prove that a prediction is correct. Explanations are therefore best presented as aids to user judgement rather than as verification.

2.0.6 Deployed tools for farmers

Several systems have moved beyond the laboratory. Ramcharan *et al.* (2017) developed a cassava disease detector using transfer learning and later deployed it as an offline mobile application evaluated on real field images, noting a marked drop in performance compared with test data (Ramcharan *et al.*, 2019). Commercial applications such as Plantix provide photo-based diagnosis to millions of users across many crops and languages (Plantix, 2024), but as proprietary products they do not publish their models, evaluation protocols or behaviour on unsupported inputs, so their accuracy and safety cannot be independently assessed. Academic prototypes, in contrast, are commonly published as notebooks or single-crop demonstrations without an interface, uncertainty handling, treatment guidance or local-language support.

2.1 Critical Analysis and Gap Identification

Table 1 contrasts representative studies against the properties a practical tool for Nepali farmers requires. Four limitations recur across the literature.

Table 1: Comparison of representative plant disease detection studies and systems

Study / system	Data and scope	Reported result	Field test	Rejects unknown crops	Explanations	Usable tool / local language
Mohanty, Hughes and Salathé (2016)	PlantVillage, 14 crops, 38 classes	99.35% (lab split)	31.4% on other-condition images	No	No	No / No
Ferentinos (2018)	87,848 images, 25 plants, 58 classes	99.53%	Poor with lab-only training	No	No	No / No
Too <i>et al.</i> (2019)	PlantVillage, 38 classes	99.75% (DenseNet-121)	Not tested	No	No	No / No
Singh <i>et al.</i> (2020) PlantDoc	2,598 field images, 13 crops, 27 classes	Up to 31% gain vs lab-only training	Yes (benchmark)	No	No	No / No
Ramcharan <i>et al.</i> (2017; 2019)	Cassava, 5 classes	93% (test); lower on field video	Yes	No	No	Yes, offline app / English
Plantix (2024)	Many crops (proprietary)	Not published	Not published	Not published	No	Yes / multilingual
CropSense AI (this project)	10 crops, 51 classes, lab + field	Independent field test (Section 6.2)	Yes, PlantDoc	Yes, measured	Grad-CAM	Web + Android / English + Nepali

- **Accuracy is reported on the wrong distribution.** Most studies evaluate on random splits of the same curated laboratory dataset, producing accuracies near 99% that the authors' own field experiments contradict (Mohanty, Hughes and Salathé, 2016; Ferentinos, 2018). The real-world performance of published models is rarely measured on independent field data.
- **Models do not know what they do not know.** Closed-set classifiers are almost never tested on crops outside their training set, although a farmer will inevitably photograph one. Open-set techniques exist in the machine-learning literature (Lee *et al.*, 2018; Geng, Huang and Chen, 2021) but have seen little application or honest evaluation in agricultural tools.
- **Explanations are shown but not qualified.** Grad-CAM is often presented as evidence that a model is right, contrary to evidence that saliency maps can be misleading (Adebayo *et al.*, 2018).
- **Research rarely reaches the intended user.** Prototypes seldom combine a model with actionable, safety-conscious treatment guidance, a usable interface on the devices farmers own, and content in the local language; commercial tools that do so are closed and unevaluable.

The gap addressed by this project is therefore **practical and evaluative**: there is no openly documented, explainable, bilingual (English–Nepali) crop disease tool that couples a modern transfer-learning model with explicit uncertainty and unknown-crop handling, and that is honestly evaluated on independent field photographs and on untrained crops rather than only on a laboratory split.

2.2 Project Justification

CropSense AI responds to each shortcoming directly. To narrow the domain gap it trains on laboratory and cleaned field images together, adds augmentations that simulate phone-camera artefacts and crops the leaf from cluttered scenes before classification. To avoid confidently wrong answers on unfamiliar inputs it combines a trained Unknown class, softmax uncertainty rules and feature-space distance to class centroids, and reports two distinct outcomes: “Crop Not Supported” and “Not Identified”. To support rather than replace user judgement it presents Grad-CAM heatmaps with explicit wording that they do not confirm the diagnosis. To reach users it delivers bilingual guidance, structured treatment information with pre-harvest intervals and safety warnings, and web and Android clients backed by a free personal account that keeps a record of every check. Finally, it measures the deployed system on an independent field benchmark and on untrained crops, producing evidence about exactly the properties the literature leaves unmeasured.

The project is relevant beyond academia. Agricultural extension offices, agrovets and non-governmental programmes in Nepal could use such a tool to triage cases and standardise first-line advice, while the evaluation results indicate what level of data collection and validation would be required before responsible large-scale use.

3.0 Project Aims, Objectives, and Scope

This section converts the problem and gap into a plan of action.

3.1 Project Aim

The aim of this project is to design, develop, deploy and critically evaluate an explainable, bilingual (English–Nepali) deep-learning system that identifies crop leaf diseases from smartphone photographs, signals uncertainty and unsupported crops, and provides actionable guidance for Nepali farmers.

3.2 Project Objectives

- **Objective 1 (Research):** Critically review at least 30 academic and professional sources on image-based plant disease detection, transfer learning, domain shift, open-set recognition and explainability, and identify a research gap, by the end of June 2026.
- **Objective 2 (Data):** Assemble, clean and document a dataset covering at least 10 crops and 50 disease/healthy classes, combining laboratory images with real field images and an Unknown class, with duplicate and cross-class-collision removal logged, by the end of August 2026.
- **Objective 3 (Model):** Train a transfer-learning classifier that fits CPU-only free hosting (under 10 million parameters), using class-imbalance handling and field-realism augmentation, checkpointed on validation macro-F1, together with calibrated open-set statistics, by 6 September 2026.
- **Objective 4 (System):** Implement a REST inference API with input validation, leaf pre-check, unknown-crop rejection and Grad-CAM, and user accounts with a server-side prediction history, and web and Android clients presenting results, confidence bands and treatment guidance in English and Nepali, by 11 September 2026.
- **Objective 5 (Deployment and testing):** Deploy the system publicly at zero hosting cost and verify it with automated tests and API edge-case tests, with all automated tests passing, by 15 September 2026.
- **Objective 6 (Evaluation):** Evaluate the deployed model on an independent real-world benchmark, reporting top-1/top-5 accuracy, per-class results, selective-prediction accuracy, untrained-crop rejection rate and response time, and evaluate the system against its objectives, by 15 September 2026.

The objectives were fixed at the start of the project. After the first public deployment, and within the same design-science cycle, the artefact was extended between 16 and 24 September 2026 with compulsory user accounts, a hosted database and a prediction history shared by the web and Android clients (Sections 5.2.5, 5.2.6 and 6.1). These additions are evaluated alongside the original objectives in Section 7.

3.3 Project Scope

Table 2: Project scope: inclusions and exclusions

In scope	Out of scope
Single-leaf images of 10 crops: apple, banana, citrus, cucumber, grape, maize, mango, potato, rice and tomato (51 conditions)	Other crops, whole-plant or field-canopy imagery, drone and satellite imagery
Classification of one dominant condition per image, with uncertainty and unsupported-crop outcomes	Multiple simultaneous diseases, severity grading, nutrient deficiencies, pest counting
Grad-CAM visual explanation of each prediction	Pixel-level lesion segmentation
General symptoms, cause, prevention and treatment guidance with safety disclaimers in English and Nepali	Prescriptive, registered pesticide recommendations or certified agronomic advice
Responsive web application, Android application (APK), public REST API	iOS App Store / Google Play publication; offline on-device inference
User accounts (sign-up, sign-in, signed session tokens) and a prediction history in a hosted PostgreSQL database, shared by the web and Android clients	Password reset, email verification, self-service account deletion, administrator dashboard and usage analytics
Technical evaluation on public benchmark data	Field trials or usability studies with farmers (no human participants)

The exclusion of human-participant studies was a deliberate decision that kept the project within the ethical approval and time constraints of an undergraduate project; its consequences are discussed in Sections 8.1 and 8.2.

4.0 Project Design

This section describes how the project was carried out: the overall research approach, the methods used for each objective, the tools, and why these choices were made.

4.1 Design and Methods

The project follows a **design science research** approach, in which knowledge is produced by building an artefact that addresses a relevant problem and rigorously evaluating it (Hevner *et al.*, 2004). Within that frame, the machine-learning work followed the phases of CRISP-DM (business understanding, data understanding, data preparation, modelling, evaluation and deployment) (Wirth and Hipp, 2000), and the software was developed iteratively in short, testable increments in the spirit of agile development (Beck *et al.*, 2001). Figure 1 summarises the resulting six-stage cycle; findings from evaluation and testing repeatedly fed back into data preparation, the model and the application.

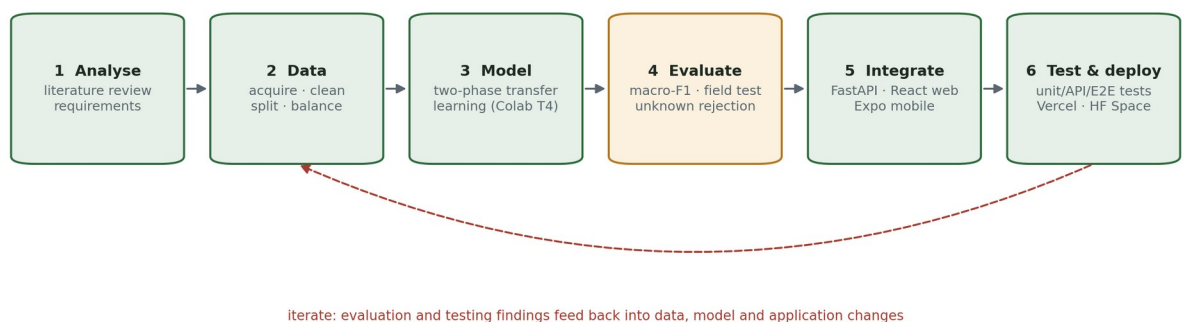


Figure 1: Iterative design-science development cycle used in the project

The project is primarily **quantitative and experimental** for the model, and **constructive** for the software. The specific methods used for each objective are listed in Table 3.

Table 3: Methods mapped to project objectives

Objective	Method(s)	Data / evidence	Output
O1 Research	Systematic keyword search (Google Scholar, IEEE Xplore, ScienceDirect); thematic critical synthesis	Peer-reviewed papers, benchmark and product documentation	Literature review, gap (Section 2)
O2 Data	Secondary data collection; automated cleaning (perceptual hashing, collision detection); manual visual review; stratified splitting	Public lab image collections; web-sourced field images	38,348 lab + 1,593 field images, cleaning log
O3 Model	Controlled training experiments: two-phase transfer learning, imbalance handling, augmentation; macro-F1 checkpointing; centroid-based OOD calibration	Train/validation splits on Google Colab (NVIDIA T4)	Weights, class list, OOD statistics
O4 System	Iterative, test-driven development; component-based UI; REST API design; accessibility guidelines (W3C, 2023)	Code, user stories, design specifications	FastAPI service, React web app, Expo app

Objective	Method(s)	Data / evidence	Output
O5 Deploy & test	Unit and component tests; API black-box edge-case testing; build and dependency checks; deployment smoke tests	Vitest, scripted HTTP tests, expo-doctor	Test results (Section 6.2)
O6 Evaluation	Independent external benchmark evaluation through the deployed API; near-duplicate filtering; selective-prediction analysis; latency measurement	PlantDoc test split (Singh <i>et al.</i> , 2020)	Metrics, figures, critical evaluation (Sections 6.2 and 7)

Data required. Training required labelled leaf images for each target class in two styles, plain-background laboratory images for learning disease signatures and cluttered field images for robustness, plus images of non-target leaves to represent the Unknown class. Evaluation required images that were never used in training; because the notebook's own held-out predictions were not preserved (Section 6.2.4), an external benchmark was adopted. Content data (symptoms, causes, management and treatment) were compiled from general agronomic guidance and translated into Nepali. No personal or human-participant data were required.

Tools. Table 4 lists the principal technologies.

Table 4: Principal tools and technologies

Layer	Technology (version)	Role
Model training	Python 3, TensorFlow/Keras 2.21, scikit-learn, OpenCV, Google Colab (T4 GPU)	Data pipeline, transfer learning, metrics
Inference API	FastAPI 0.141, Uvicorn, Pillow 12.3, NumPy 2.2, OpenCV 5.0	REST endpoints, preprocessing, Grad-CAM
Accounts and data	PostgreSQL 16 (local, Docker) and 18 (Neon, hosted), psycpg2 2.9.12, bcrypt 5.0.0, python-jose 3.5.0 (JWT), expo-secure-store	User accounts, signed sessions, prediction history
Web client	React 19.2, Vite 8, React Router 7, Axios 1.18, CSS Modules	Responsive single-page application
Mobile client	Expo SDK 57, React Native 0.86, expo-image-picker	Android application (APK via EAS Build)
Testing & quality	Vitest 3.2, React Testing Library, ESLint, expo-doctor, Python requests scripts	Automated and scripted tests
Local environment	Docker Compose (backend, web/nginx, PostgreSQL 16, Adminer)	Reproducible full-stack runs
Hosting	Hugging Face Space (Gradio SDK, CPU inference, Git LFS), Vercel, Neon (serverless PostgreSQL), Expo EAS	Zero-cost public deployment
Collaboration	Git and GitHub	Version control and CI deployment trigger

4.2 Justification of Methods

Design science over purely experimental research. The problem is practical: farmers need a usable tool, not only a better accuracy figure. Design science legitimises building the artefact as the research contribution while still demanding rigorous evaluation (Hevner *et al.*, 2004). A purely experimental comparison of architectures on PlantVillage would have added little, because accuracy on that data is already saturated (Section

2.0.1). A survey or interview-based study was rejected because it would describe the need without testing a solution, and would have required ethical approval and access to farmers that were not available within the project timeframe.

Transfer learning with EfficientNetB2. Training a CNN from scratch on about 40,000 images would underfit or overfit, whereas ImageNet pre-training transfers generic visual features (Zhuang *et al.*, 2021). Table 5 compares candidate backbones. MobileNetV2 and EfficientNetB3 were explored in early iterations (their configuration folders are retained in the repository). EfficientNetB2 was chosen as the best balance: it is considerably more accurate on ImageNet than MobileNetV2 while having about a tenth of VGG16's parameters, it uses a 224×224 input matching the source images, and its 8.5-million-parameter model fits comfortably in the memory of free CPU hosting. B3 was set aside because its larger 300×300 input increases computation roughly quadratically for little expected gain on 224-pixel source images.

Table 5: Comparison of candidate CNN backbones (ImageNet figures from Keras (2026))

Backbone	Parameters	ImageNet top-1	Default input	Assessment for this project
VGG16	138.4 M	71.3%	224	Too large for free CPU hosting; slow
ResNet50 (He <i>et al.</i> , 2016)	25.6 M	74.9%	224	Accurate but nearly three times larger than B2
MobileNetV2 (Sandler <i>et al.</i> , 2018)	3.5 M	71.3%	224	Very light; lower accuracy (baseline iteration)
EfficientNetB2 (Tan and Le, 2019)	9.2 M (7.8 M without top)	80.1%	260 (used at 224)	Selected: best accuracy per parameter within budget
EfficientNetB3 (Tan and Le, 2019)	12.3 M	81.6%	300	Higher cost; explored and set aside

Two-phase fine-tuning. Training only the new classification head first, then unfreezing most of the backbone at a learning rate two orders of magnitude lower, prevents large gradients from a randomly initialised head destroying pre-trained features, a standard and well-supported practice (Too *et al.*, 2019). The Adam optimiser (Kingma and Ba, 2015), a cosine learning-rate decay (Loshchilov and Hutter, 2017) and mixed-precision training (Micikevicius *et al.*, 2018) were used because they are robust defaults that shorten training on limited free GPU time.

Macro-F1 rather than accuracy. The laboratory data are highly imbalanced (from 23 to 2,384 images per class). Overall accuracy rewards majority classes, whereas macro-F1 weights every class equally (Sokolova and Lapalme, 2009), so checkpointing on validation macro-F1 prevents the saved model from ignoring rare diseases (Buda, Maki and Mazurowski, 2018).

Combining softmax rules with feature-space distance for unknown inputs. Softmax confidence alone cannot reject a confidently misclassified untrained crop (Hendrycks and Gimpel, 2017; Guo *et al.*, 2017). Feature-space methods are reported to be stronger (Lee *et al.*, 2018), but full Mahalanobis estimation over a 512-dimensional embedding with some very small classes is numerically fragile. Cosine similarity to L2-normalised class centroids was therefore chosen as a simple, computationally trivial

approximation that reuses the forward pass already needed for classification, with its threshold calibrated to accept 95% of known validation leaves. Its actual effectiveness is measured in Section 6.2.4.

Grad-CAM for explanation. Grad-CAM needs no retraining or architectural change and produces class-specific heatmaps from the existing network (Selvaraju *et al.*, 2017). Alternatives such as LIME or SHAP require many forward passes per image, which would multiply CPU inference time on free hosting. Following Adebayo *et al.* (2018), heatmaps are presented as a guide rather than proof.

Independent benchmark evaluation. Re-running training to regenerate a held-out split was not feasible within the remaining time and free GPU quota, and a random lab split would in any case overstate field performance (Section 2.0.3). Evaluating the deployed endpoint on PlantDoc, a public benchmark of real field photographs (Singh *et al.*, 2020), measures what a user would actually experience, including preprocessing and the unknown gate, and provides untrained-crop images for rejection testing. The cost is a smaller, noisier test set covering only the overlapping classes, which is acknowledged in the evaluation.

Application stack. FastAPI was preferred to Flask for its automatic request validation and OpenAPI documentation, which also serves as a testable API contract; it is also asynchronous, which suits upload handling. React with Vite was chosen for a fast, component-based single-page application with a mature testing ecosystem. Expo/React Native allowed the Android client to share JavaScript skills and normalisation logic with the web client and to be built into an installable APK in the cloud without a local Android SDK. A thin-client architecture, in which both clients call the same API, keeps all inference logic in one place so that threshold changes apply consistently everywhere. Hugging Face Spaces and Vercel were selected because they offer HTTPS hosting without payment details; alternatives such as Render and Koyeb free tiers offer only 512 MB of memory, insufficient for TensorFlow, while major cloud providers require a credit card.

Accounts, tokens and a relational database. When accounts were added, three choices had to be justified. PostgreSQL was chosen because the data are naturally relational (one user owns many predictions), because ownership can be enforced with a foreign key and a simple query, and because the same engine runs locally in Docker and, free of charge and without a payment card, as a Neon serverless database in production. Sessions use signed JSON Web Tokens (Jones, Bradley and Sakimura, 2015) rather than server-side sessions: a token can be verified without a database lookup, which matters on a free Space that sleeps and restarts, and it expires after 24 hours. Passwords are stored only as bcrypt hashes, an adaptive, salted scheme designed to stay slow as hardware improves (Provos and Mazières, 1999). On Android the token is kept in expo-secure-store, which is backed by the Android Keystore, rather than in plain storage, because anyone holding it can read that user's history. Accounts were made compulsory, rather than optional, so that every check has an owner and the same history appears on both clients; the cost of this choice, an extra step before the first diagnosis, is evaluated in Section 7.

5. Artefact Design

This section gives an objective description of what the artefact must do, the constraints it operates under, and how its components are structured.

5.1 Design Specifications

Requirements were derived from the problem statement, the gap analysis and the needs of the intended users: farmers and extension workers using low-to-mid-range Android phones or browsers, with variable connectivity and literacy in Nepali or English. Figure 2 shows the main use cases and the three actors: the farmer or extension worker, the model service and the accounts database.

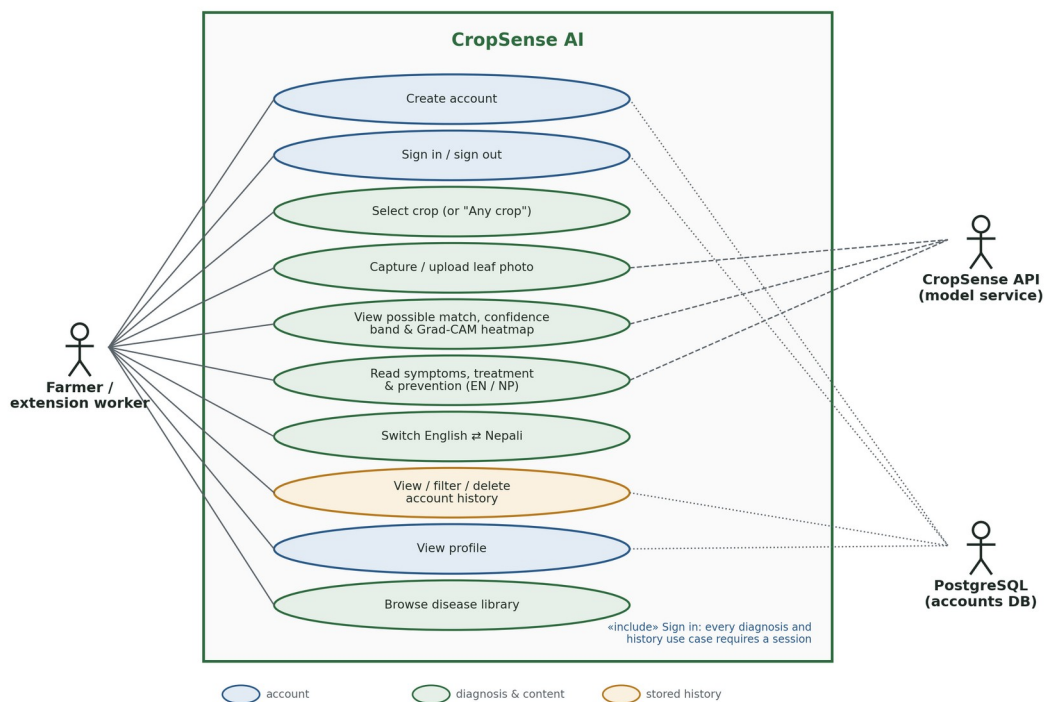


Figure 2: Use case diagram of CropSense AI (all diagnosis and history use cases require a signed-in session)

Table 6: Functional requirements

ID	Functional requirement	Priority	Status
FR1	Accept a leaf image by camera capture or upload (JPG, PNG, WEBP) on web and Android	Must	Met
FR2	Let the user select one of 10 supported crops or "Any crop"	Must	Met
FR3	Reject empty, corrupt and non-image files with a clear message	Must	Met (one size defect, Section 6.2.3)
FR4	Detect images that are not leaves before classification	Should	Met
FR5	Predict the most likely of 51 conditions and return the top-5 alternatives with confidences	Must	Met

ID	Functional requirement	Priority	Status
FR6	Report “Crop Not Supported” or “Not Identified” instead of forcing a class when the input is unfamiliar or uncertain	Must	Implemented; effectiveness limited (Section 6.2.4)
FR7	Warn when the predicted crop differs from the selected crop	Should	Met
FR8	Show a Grad-CAM heatmap beside the original photo	Must	Met
FR9	Present a High / Moderate / Uncertain confidence band rather than a bare percentage	Should	Met
FR10	Provide symptoms, cause, management, prevention and structured treatment options with dose, pre-harvest interval and safety band	Must	Met
FR11	Provide all user-facing content in English and Nepali with an instant language toggle	Must	Met
FR12	Save every check to the user's account and show the history on web and Android, with search, filters, details and deletion	Should	Met
FR13	Provide a browsable disease library	Could	Met (web)
FR14	Expose a documented public REST API	Should	Met (/docs)
FR15	Let a user create an account (username of at least 3 characters, valid email, password of at least 8 characters) and sign in and out	Must	Met
FR16	Restrict every screen except sign-in and create-account to signed-in users, and show a profile with the number of saved checks	Should	Met
FR17	Allow a user to read or delete only their own saved checks	Must	Met (HTTP 403 otherwise)

Table 7: Non-functional requirements and constraints

ID	Non-functional requirement / constraint	Target
NFR1 Performance	Response time for one prediction including Grad-CAM	< 10 s on free hosting; < 3 s locally
NFR2 Resource	Model and service must fit free CPU hosting memory	< 10 M parameters; no GPU required
NFR3 Cost	Hosting and distribution	£0 recurring cost; no payment card
NFR4 Usability	Core task (photo to result) reachable from the home page in few steps; plain language	≤ 3 user actions
NFR5 Accessibility	Keyboard operable, screen-reader announcements, visible focus, reduced motion, 44 px targets, text not colour alone	WCAG 2.2 AA principles (W3C, 2023)
NFR6 Responsible output	Results framed as “possible match”; treatment disclaimer on every recommendation; explanation caveat	Present on every result
NFR7 Privacy	Minimal personal data: username, email and password hash; no full-size photo stored, only a 160×120 thumbnail and the heatmap of each check; history visible only to its owner	Data minimisation (Data Protection Act 2018); HTTP 403 on another user's record
NFR8 Robustness	Invalid input returns HTTP 400, never HTTP 500; a database or token problem must never stop a diagnosis	No unhandled errors in tests
NFR9 Maintainability	Single inference implementation shared by all clients; pinned dependencies; automated tests	All tests pass
NFR10 Portability	Runs locally via Docker Compose and on managed hosting from the same code	One code base
NFR11 Security	Passwords hashed with bcrypt; signed tokens that expire after 24 hours; token in the Android Keystore on mobile; TLS to the database; secrets kept outside the code	No plaintext password or secret in the repository or database

5.2 Artefact's Structure / Architecture / Framework

CropSense AI uses a three-tier, thin-client architecture (Figure 3). Presentation is handled by two clients: a React single-page application served as static files from Vercel's content delivery network, and an Android application. Both sign the user in and then send the same multipart POST /predict request, carrying the session token, over HTTPS to a single FastAPI service hosted on a Hugging Face Space. The service contains the whole inference pipeline, the knowledge base and the account and history endpoints, and returns one JSON document containing everything the clients need to render a result in either language. The data tier is a PostgreSQL database holding accounts and saved checks, hosted on Neon in production. No client performs inference, so a change to a threshold or the model changes behaviour consistently everywhere, and the database sits off the critical path: if it is unreachable, sign-in and history stop working but diagnosis does not.

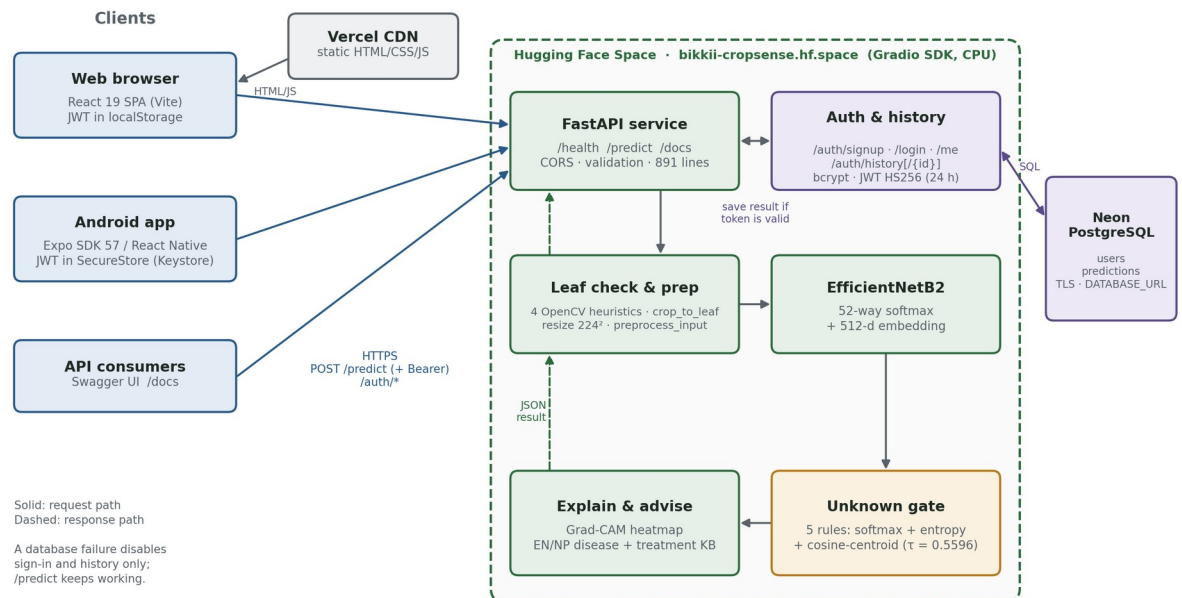


Figure 3: System architecture of the deployed CropSense AI artefact, including the accounts database

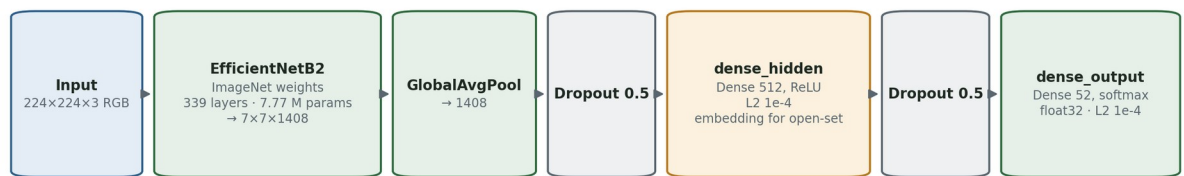
Table 8: Architectural components and responsibilities

Component	Responsibility	Key implementation details
Web client	Sign-in, capture/upload, crop selection, result display, account history, library, profile, language toggle	React 19; one API module (src/lib/api.js) that adds the Bearer token; pure normalisation layer (normalize.js) deriving confidence bands; route guard (RequireAuth); account history mirrored into localStorage as a cache
Android client	Sign-in, camera/gallery capture, crop chips, bilingual result view, history, profile	Expo SDK 57; token in expo-secure-store (Android Keystore); 40 s prediction timeout; image quality 0.8
FastAPI service	Validation, orchestration, response assembly	backend/main.py (891 lines); CORS enabled; NumPy-to-JSON conversion; history saving wrapped so it can never block prediction

Component	Responsibility	Key implementation details
Auth & history	Sign-up, sign-in, profile, per-user history	auth.py: bcrypt, JWT HS256 with 24-hour expiry; five /auth routes; ownership check on every read and delete
Database	Persist users and saved checks	PostgreSQL: users and predictions tables (Figure 9); Neon over TLS in production, Docker locally; psycopg2 driver
Leaf pre-check	Reject obvious non-leaf images	Four OpenCV heuristics (leaf-colour ratio, grey-level variance, edge density, aspect ratio); at least three must pass
Leaf cropping	Focus the model on the leaf in cluttered photos	Excess-green and diseased-tissue colour mask, morphology, largest contour, 10% padding; returns the original image whenever the mask is implausible
Classifier	Predict 52 classes and a 512-dimensional embedding	EfficientNetB2 backbone + dense head; one forward pass yields both outputs
Unknown gate	Decide between a result, “Not Identified” and “Crop Not Supported”	Five OR-ed rules (Table 9)
Explainer	Visual attribution	Grad-CAM on the last 4-D backbone feature map; JET colour map blended 60/40
Knowledge base	Bilingual guidance	disease_info.json and treatments.json: 51 diseases, 122 treatment entries

5.2.1 Model architecture

The classifier (Figure 4) is an ImageNet-pre-trained EfficientNetB2 backbone without its top, followed by global average pooling, dropout of 0.5, a 512-unit ReLU layer named dense_hidden, a second dropout of 0.5 and a 52-way softmax output. Both dense layers use L2 regularisation of 1×10^{-4} . The hidden layer is deliberately named because its activations serve as the embedding for open-set rejection, and the output layer is kept in 32-bit precision because softmax is numerically unstable under the mixed-precision policy used in training. The complete model has 8,516,653 parameters, of which 7,768,569 are in the backbone.



Total 8,516,653 parameters. Phase 1 trains only the head (748,084 parameters); Phase 2 unfreezes all backbone layers after the first 100.

Figure 4: Classifier architecture (EfficientNetB2 backbone with a regularised dense head)

5.2.2 Inference pipeline and decision logic

Figure 5 traces a request through the service. It has three distinct exits that avoid showing a disease result: an HTTP 400 error for unreadable input, a “Not a Leaf” result from the pre-check, and one of two unknown outcomes from the decision gate. All non-error outcomes share one response schema, so clients never branch on HTTP status to discover that the model declined.

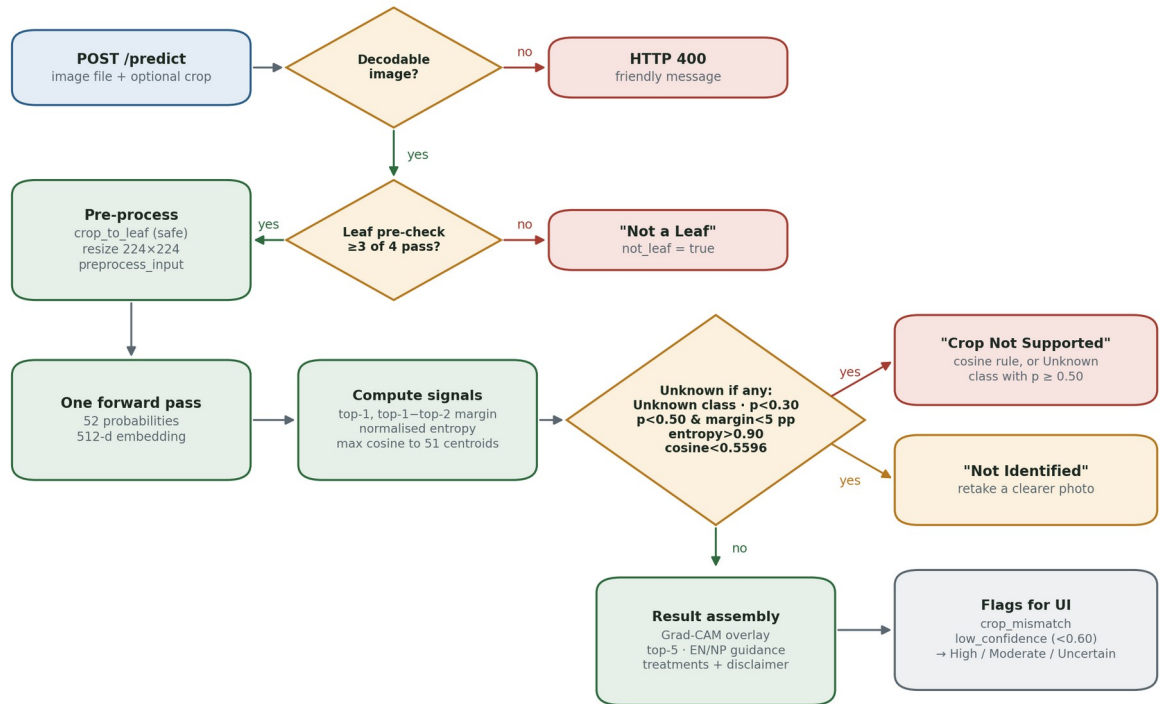


Figure 5: Request flow and decision logic of the /predict endpoint

For a softmax output $\mathbf{p}$ over $C = 52$ classes, the gate computes the top-1 probability, the margin between the two largest probabilities, the normalised entropy and the open-set score:

$$H(\mathbf{p}) = - \sum_i p_i \cdot \log(p_i) / \log(C) \quad \text{normalised entropy: } 0 = \text{certain}, 1 = \text{uniform}$$

$$s(\mathbf{x}) = \max_k \cos(\mathbf{e}(\mathbf{x}), \mu_k) \quad \mathbf{e}(\mathbf{x}): \text{L2-normalised dense_hidden embedding}$$

$$\mu_k = \text{normalised mean training embedding of class } k \text{ (51 known classes; Unknown excluded)}$$

$$\tau = 5\text{th percentile of } s(\mathbf{x}) \text{ over known validation leaves} = 0.5596 \text{ (accepts 95\%)}$$

Table 9: Rules of the unknown gate implemented in the backend

#	Rule (any one triggers)	Purpose	Reported outcome
1	Top-1 class is Unknown__Unknown	The model itself recognises a non-target leaf	Crop Not Supported if $p \geq 0.50$, else Not Identified
2	Top-1 probability < 0.30	Flat uncertainty	Not Identified
3	Top-1 < 0.50 and margin < 5 percentage points	Near-tie between classes	Not Identified
4	Normalised entropy > 0.90	Probability spread across many classes	Not Identified
5	Open-set score $s(\mathbf{x}) < \tau = 0.5596$	Confident prediction on an unfamiliar leaf	Crop Not Supported

Clients then derive the user-facing confidence band with a pure function: any backend doubt flag (unknown, not a leaf, crop mismatch or confidence below 60%) forces **Uncertain**; otherwise confidence of at least 80% is **High**, at least 60% is **Moderate**, and anything lower is **Uncertain**. This ordering guarantees that, for example, an 85%-confident prediction on a leaf of the wrong crop is never displayed as High confidence.

5.2.3 API and response design

Table 10: Public API endpoints

Method	Path	Purpose
GET	/health	Liveness: model loaded, backbone and number of classes
POST	/predict	Multipart form: file (required), crop_type (optional); optional Bearer token: if valid, the result is also saved to that user's history. Returns the diagnosis JSON
GET	/docs	Interactive OpenAPI (Swagger) documentation
POST	/auth/signup	Create an account (username ≥ 3 characters, valid email, password ≥ 8 characters)
POST	/auth/login	Verify the password; return a JWT (HS256, 24 h), username and email
GET	/auth/me	Profile of the signed-in user (Bearer token required)
GET	/auth/history	The user's last 50 checks, newest first, with thumbnails but without heatmaps
GET	/auth/history/{id}	One saved check in full, including its heatmap; 403 if it belongs to someone else
DELETE	/auth/history/{id}	Delete one saved check; 403 if it belongs to someone else

Table 11: Structure of the /predict response

Group	Fields
Identity	disease, disease_np, raw_class, crop_type
Confidence	confidence, low_confidence, entropy, ood_score
Decision flags	is_unknown, not_leaf, unknown_reason, unfamiliar_crop, crop_mismatch, supported_crops
Ranking	top_5_predictions[], raw_probabilities[] (all 52 classes)
Guidance (EN + NP)	description, symptoms, cause, treatment, prevention, each with an _np twin; message; disclaimer
Treatments	treatments[] with active ingredient, kind, hazard band, formulation, dose, pre-harvest interval, re-spray interval, notes; treatment_disclaimer(_np)
Explanation	gradcam_image (base64 JPEG data URI)

5.2.4 Knowledge base and interface design

Every content field in the knowledge base is stored as an English–Nepali pair, and both languages are returned in every response. Language switching is therefore instant and needs no extra request, and safety text is never the part left untranslated. The 122 treatment entries are structured rather than free text: each of the 51 diseases has a non-chemical (cultural) step listed first, followed where appropriate by chemical, biological or biopesticide options with a formulation, dose, pre-harvest interval, re-spray interval and a hazard colour band matching the product label triangle (44 blue, 18 yellow, 8 green, 1 red). A prominent disclaimer instructs users to follow the product label and confirm doses and local registration with an agrovet or agriculture office.

The interface follows a small set of principles: one primary action per screen; possible-match wording and confidence bands instead of definitive diagnoses; status conveyed by text as well as colour; large touch targets; and a visual identity reflecting Nepali agriculture. Accessibility features include a skip link, semantic landmarks, a keyboard-operable crop radio group, aria-live progress announcements, role="alert" errors, visible focus rings and support for reduced-motion preferences (W3C, 2023).

Figure 6 shows the resulting user journey, which is the same on the web and on Android. A session check comes first; the diagnosis flow then has three possible outcomes, each with its own next step for the user, and every outcome is saved to the account history.

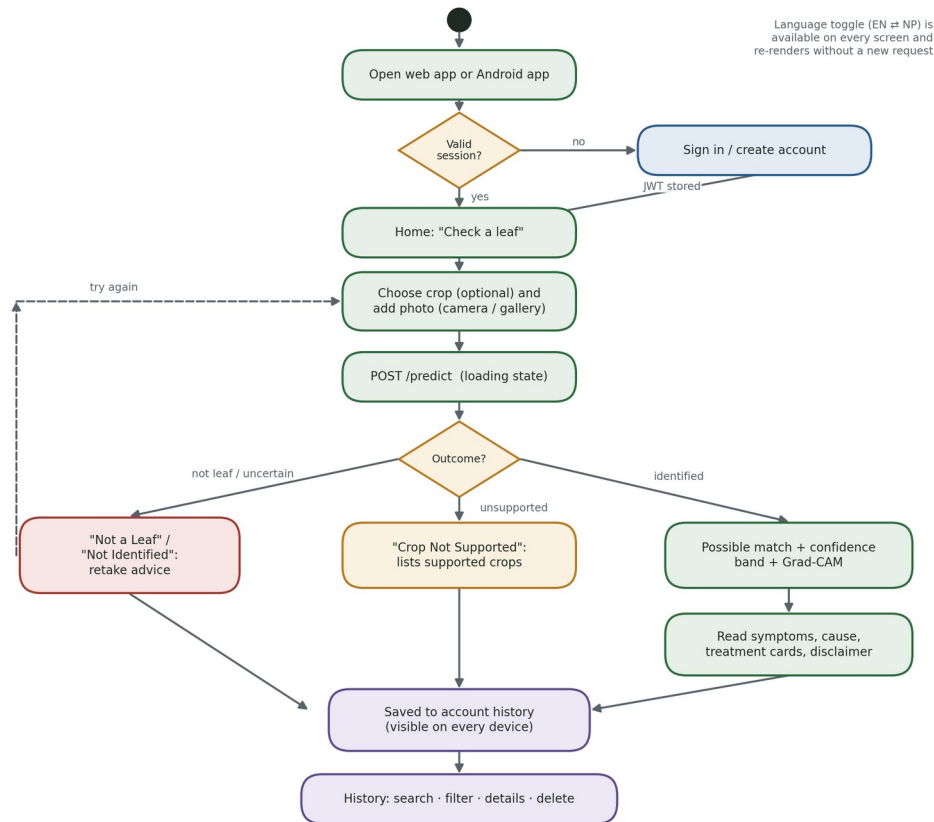


Figure 6: Activity diagram of the user journey on the web and Android clients

5.2.5 Data flow and data design

Figure 7 places the system in its environment as a context (level 0) data-flow diagram. The farmer supplies credentials, a photograph, an optional crop and a language choice, and receives a possible match, a confidence band, a heatmap, guidance and their history. The developer supplies the model artefacts and knowledge base, and the system deliberately points users to an agrovet or extension office to confirm any chemical treatment.

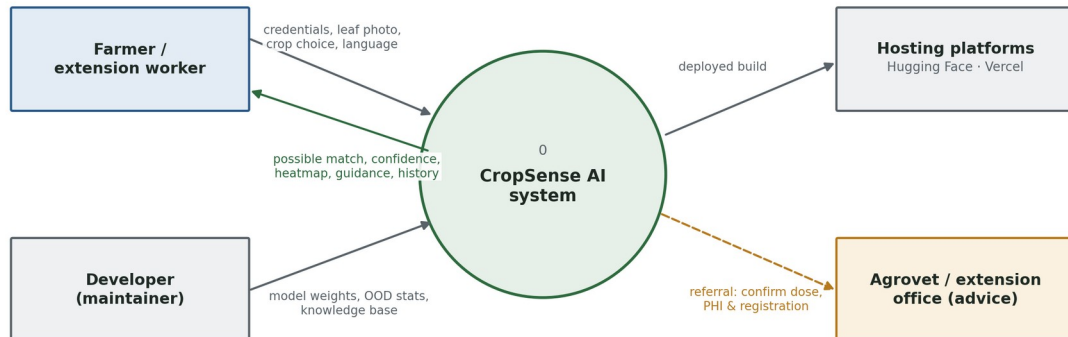


Figure 7: Context (level 0) data-flow diagram of CropSense AI

Figure 8 decomposes the system into six processes and four data stores. Two stores are files shipped with the model (D3, the weights and class centroids, which must come from the same training run; D4, the bilingual knowledge base), and two are database tables (D1 and D2). Process 5.0 only runs when the request carries a valid token, and a failure there is logged rather than returned, so the flow from 2.0 to 4.0 back to the farmer never depends on the database.

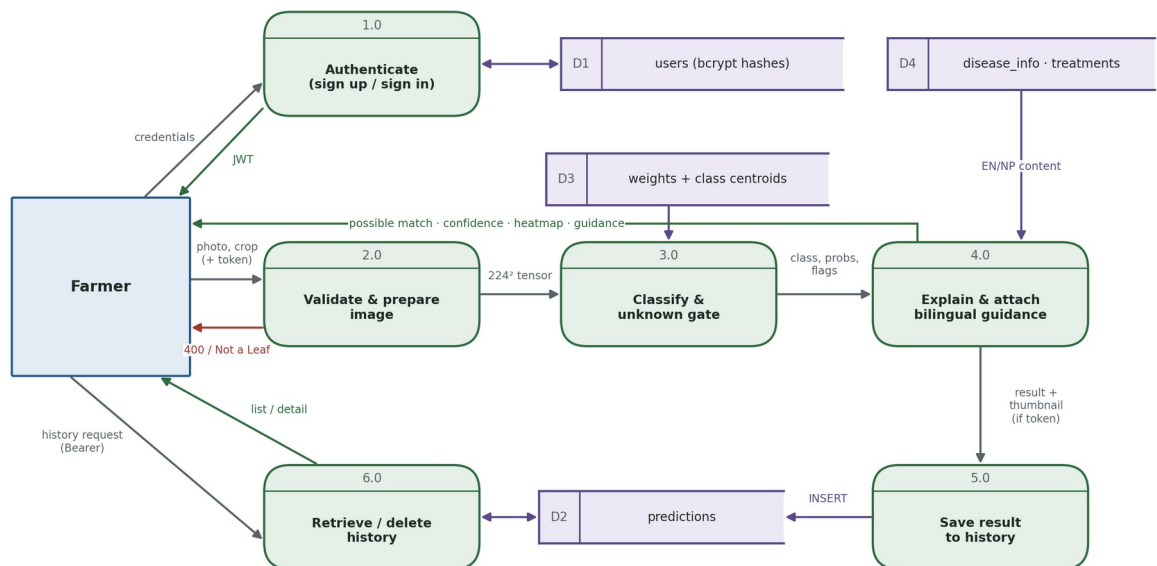


Figure 8: Level 1 data-flow diagram: processes, data stores and flows

The database schema (Figure 9; database/schema.sql, mirrored by `init_db()` in backend/db.py) has two tables in a one-to-many relationship. A prediction row stores the result in both languages, so the history can be re-rendered in either language without calling the model again, together with a 160×120 thumbnail and the Grad-CAM overlay

as base64 text. The full-size photograph is never stored. Table 12 summarises each table and the rule that controls access to it.

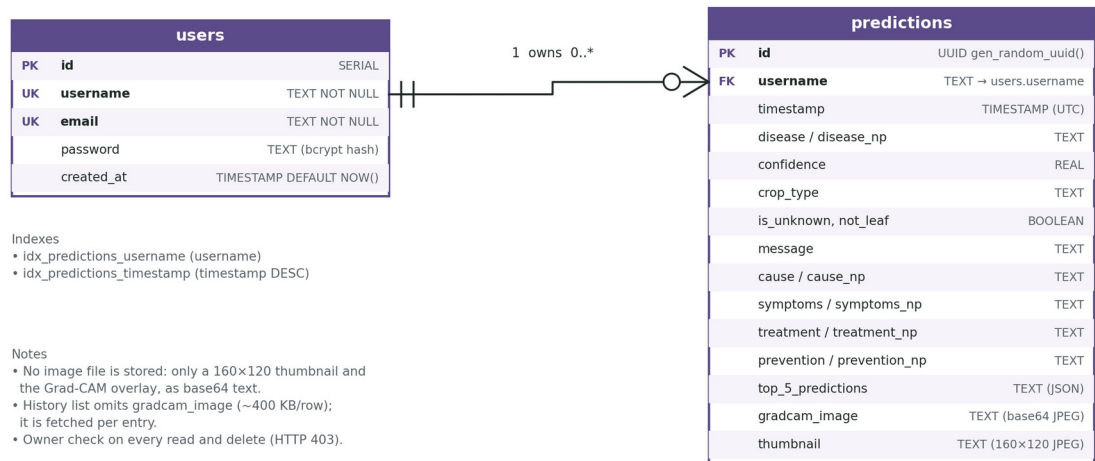


Figure 9: Entity-relationship diagram of the accounts database

Table 12: Database tables, content and access rules

Table	Keys and indexes	Content	Access rule
users	id (PK, serial); username and email unique	Username, email, bcrypt password hash, creation time (UTC)	Written at sign-up; read at sign-in and by /auth/me
predictions	id (PK, UUID); username (FK → users); indexes on username and timestamp DESC	Result in English and Nepali, confidence, flags, top-5 list, thumbnail, heatmap, timestamp (UTC)	Inserted by /predict when the token is valid; read and deleted only by the owner (HTTP 403 otherwise)

Two details of the schema design came from defects found in use. First, PostgreSQL returned timestamps without a time zone, so clients read UTC as local time and every check in Nepal appeared 5 hours 45 minutes early; the server now labels them explicitly as UTC. Second, each heatmap is about 400 KB of base64, so returning full rows made a 50-entry history roughly 19 MB and the phone timed out. The list endpoint therefore returns thumbnails only (7.9 KB for one entry in testing), and a detail view fetches the heatmap for a single check.

5.2.6 Authentication and session design

Figure 10 shows how the account endpoints and the prediction endpoint work together. The client signs in once, stores the token and sends it as a Bearer header on every later request. /predict accepts the token as optional: with a valid one the result is saved, and with a missing, expired or tampered one the diagnosis is still produced and simply not saved, because an expired session should never stop a farmer from diagnosing a leaf. The requirement to sign in is enforced by the clients, while the server enforces ownership of stored data. Table 13 lists the security controls and their known gaps.

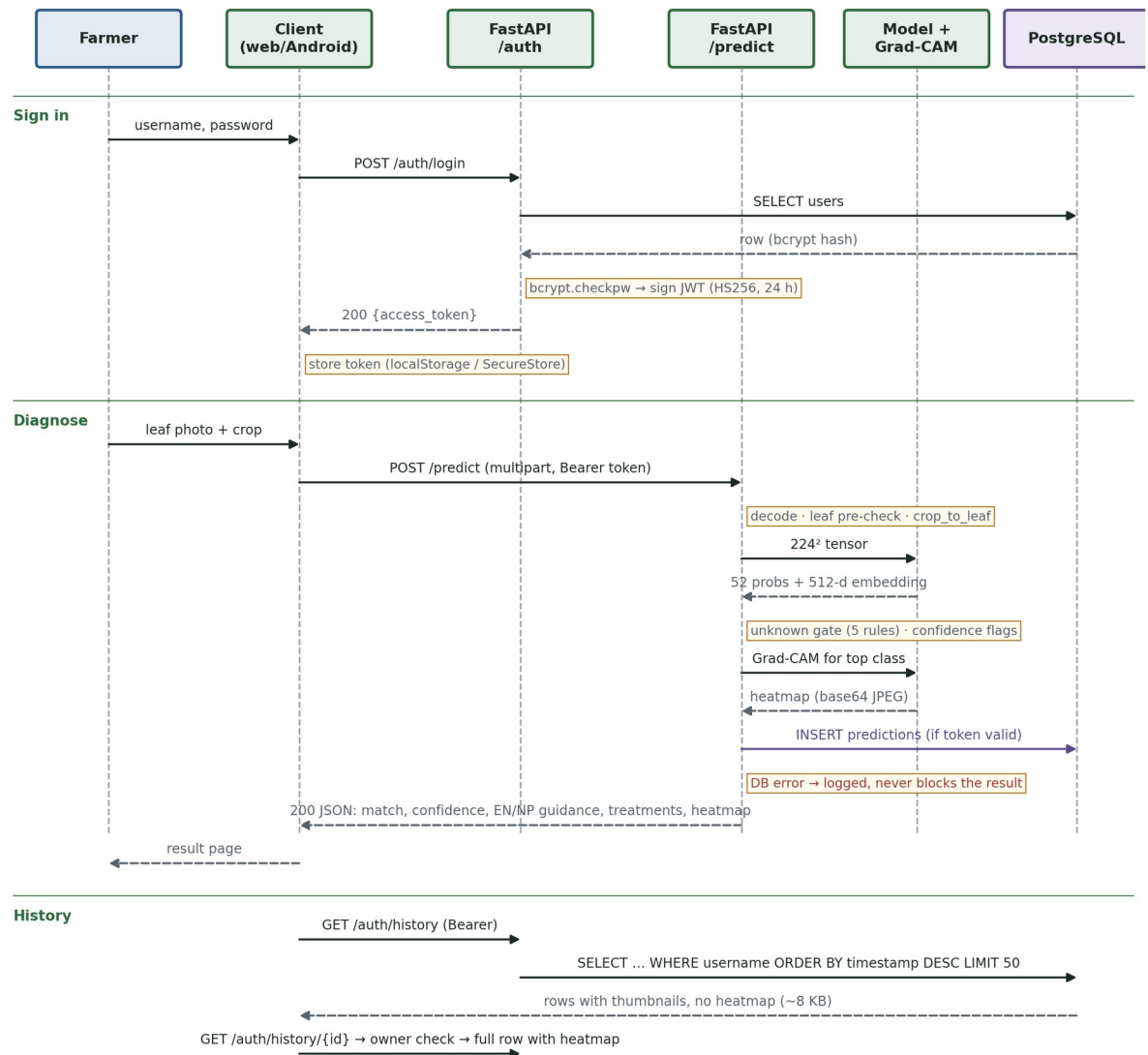


Figure 10: Sequence diagram: sign-in, diagnosis with history saving, and history retrieval

Table 13: Security controls for accounts and history

Concern	Control implemented	Known gap
Password storage	bcrypt with a per-password salt (input truncated to bcrypt's 72-byte limit); plaintext never stored or logged	No password-strength check beyond 8 characters
Session tokens	JWT HS256 signed with JWT_SECRET, 24-hour expiry; no hard-coded fallback key, so a missing secret fails closed (random key per process, with a warning)	No refresh or revocation; sign-out only deletes the client copy
Token storage on devices	Android: expo-secure-store (Keystore). Web: localStorage	localStorage is readable by any script that runs on the page (XSS exposure)
Data ownership	Every history read and delete checks that the row belongs to the token's user (HTTP 403)	No self-service account deletion or data export

Concern	Control implemented	Known gap
Transport and secrets	HTTPS to the Space; TLS (sslmode=require) to Neon; DATABASE_URL and JWT_SECRET held as Space secrets, not in git	CORS allows all origins
Brute force	Generic 'invalid username or password' message	No rate limiting or lockout: 20 wrong passwords in a row all returned 401 (Section 6.2.3)

6. Implementation

This section describes how the artefact was built and how it was validated. Development ran from June to September 2026 (Appendix A) and is recorded in the project's Git history.

6.1 Construction of the Artefact

6.1.1 Data acquisition, cleaning and preparation

Laboratory data. The laboratory dataset contains 38,348 images in 51 class folders named with a strict Crop__Disease convention. It was compiled from publicly available leaf image collections: PlantVillage-style plain-background images for apple, grape, maize, potato and tomato (Hughes and Salathé, 2015), and other public collections for banana, citrus, cucumber, mango and rice. Figures 11 and 12 show the resulting distributions. The imbalance is severe: the largest class, Maize Common rust, has 2,384 images while the smallest, Banana Yellow Sigatoka, has only 23, a ratio of about 103:1. Rice is the only crop without a healthy class.

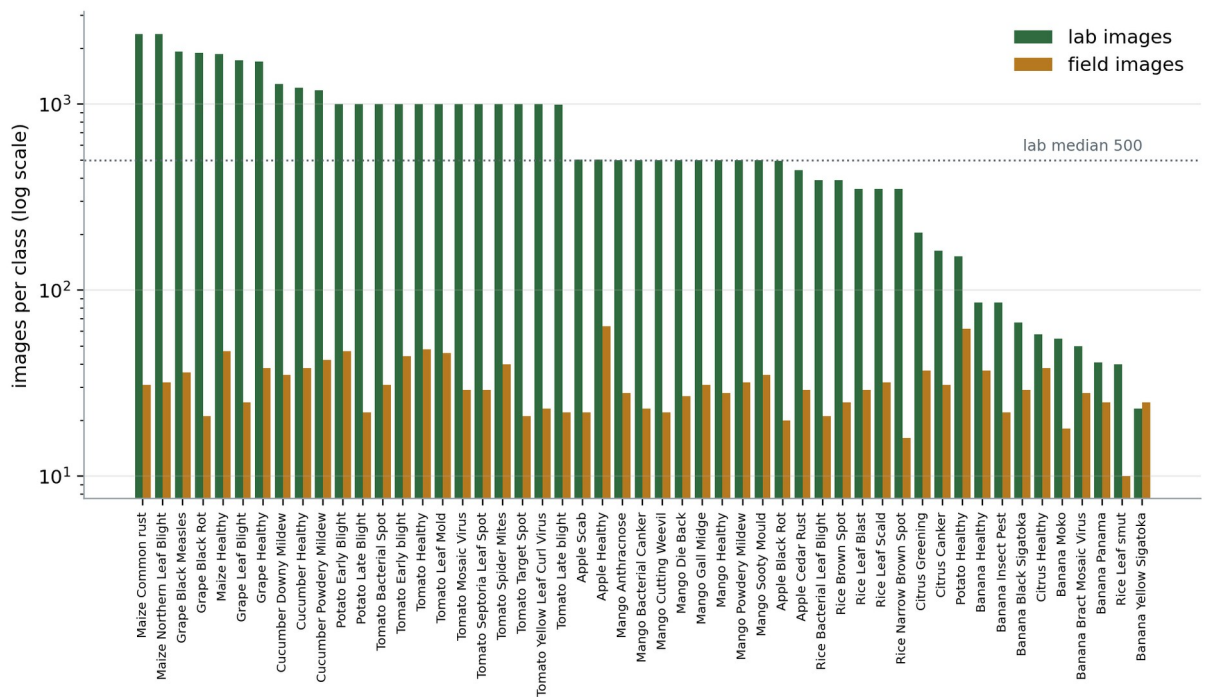


Figure 11: Images per class in the laboratory and field datasets (logarithmic scale)

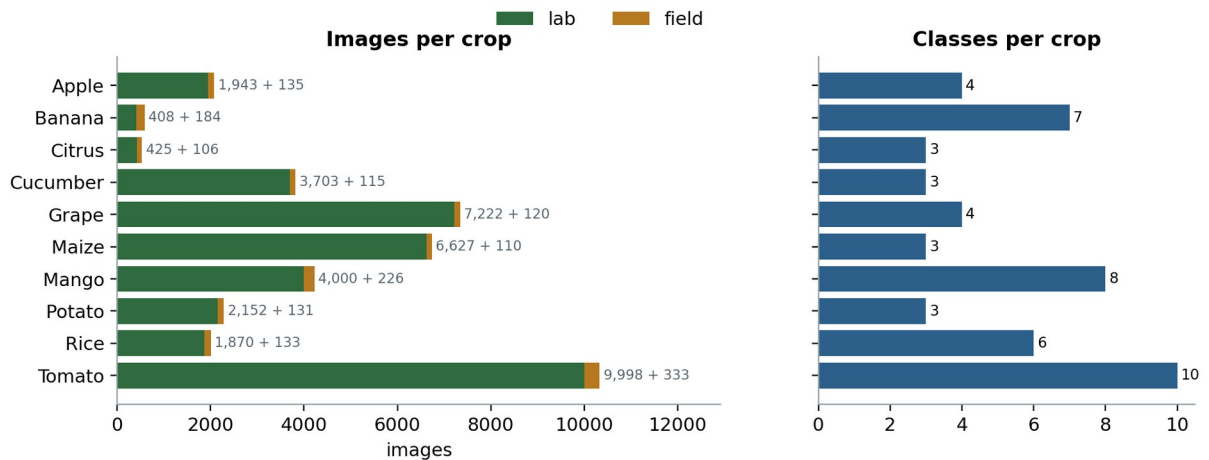


Figure 12: Images and classes per crop

Field data. Because laboratory-only models fail in the field (Section 2.0.3), a real-world dataset was built with five purpose-written scripts. Images were collected from web image search with one or more hand-written queries per class, including Latin pathogen names to avoid visual look-alikes. Cleaning removed only objective problems without relabelling: corrupt files and images below 224 pixels on the short side were deleted; **274 near-duplicates** within class folders were removed using an 8×8 average hash with a Hamming distance of at most 4 (Zauner, 2010); and **43 cross-class collisions**, the same picture appearing under two different labels, were resolved by deleting every copy, since at least one label must have been wrong. Each class was then rendered as a numbered contact sheet for manual visual review, and thin classes were re-downloaded. Every deletion was logged. The final field set contains 1,593 images across 51 classes (10–64 per class).

Unknown class. A 52nd class, Unknown__Unknown, was built up to a target of 1,000 images: up to 700 real leaves of crops outside the model (for example guava, papaya and coffee), about 200 random real-world photographs, and synthetic noise and leaf-like textures capped at 35% of the class so that filler could not dominate it.

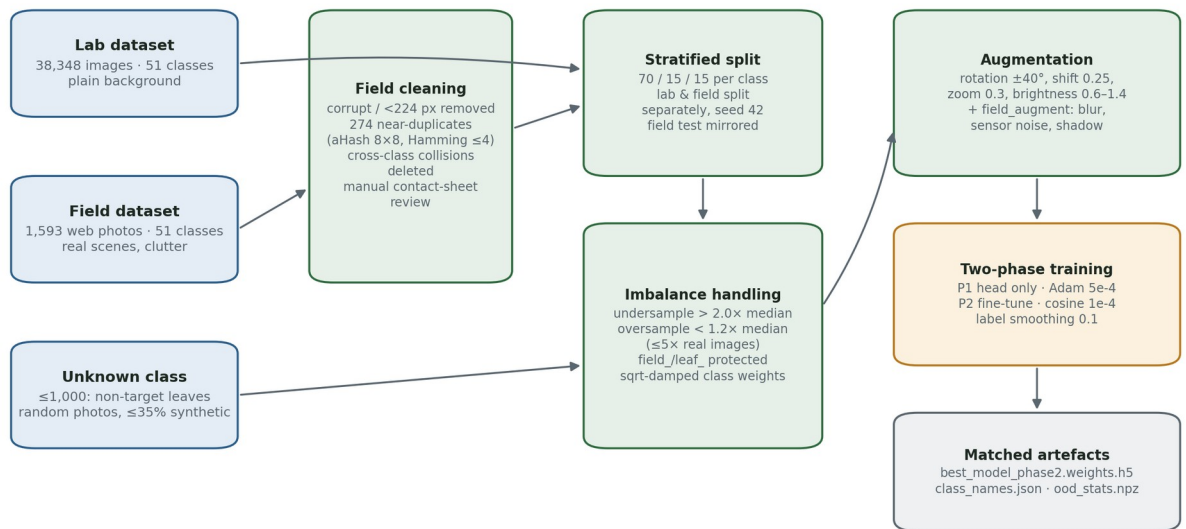
Splitting and balancing. Laboratory and field images were each split 70/15/15 into training, validation and test sets per class with a fixed random seed of 42, then merged, so every class was learned from both domains. The field test slice was also mirrored into a separate field-only test folder; classes with fewer than 20 field images (3 classes) contributed all of them to training. Imbalance was handled in three deliberately gentle layers so that corrections would not compound: majority classes were undersampled to 2.0× the median class size and minority classes oversampled to 1.2× the median (never more than five times their real images); real field photographs and real non-target leaves were protected from undersampling; and square-root-damped class weights, $w_c = \sqrt{\text{mean count} / n_c}$, were applied. An earlier linear weighting scheme (weights from 1 to 6) on top of resampling had over-corrected and destabilised training, which motivated this design. All images were resized to 224×224 once on disk to remove a CPU bottleneck in every epoch.

Table 14: Datasets used in the project

Dataset	Images	Classes	Characteristics	Role
Laboratory	38,348	51	Single leaf, plain background, controlled light; 23–2,384 per class	Learn disease signatures
Field (cleaned)	1,593	51	Real scenes: clutter, shadow, multiple leaves; 10–64 per class	Reduce domain gap; field-only test
Unknown (target)	≤ 1,000	1	Non-target leaves, random photos, ≤ 35% synthetic	Learn to decline
PlantDoc test (external)	236 (226 after de-duplication)	27 folders	Real web field photos; independent source (Singh <i>et al.</i> , 2020)	Independent evaluation only

6.1.2 Model training

Training was implemented in a Google Colab notebook (scripts/train_colab_google.ipynb) running on an NVIDIA T4 GPU, with data and checkpoints on Google Drive. Figure 13 summarises the pipeline. The notebook is resume-safe: phase-two weights and an epoch counter are written to Drive after every epoch, so a Colab disconnection resumes rather than restarts training. Image loading uses eight threads; threads are used instead of processes because forked workers would inherit identical random states and repeat the same augmentations.

**Figure 13:** Data preparation and training pipeline

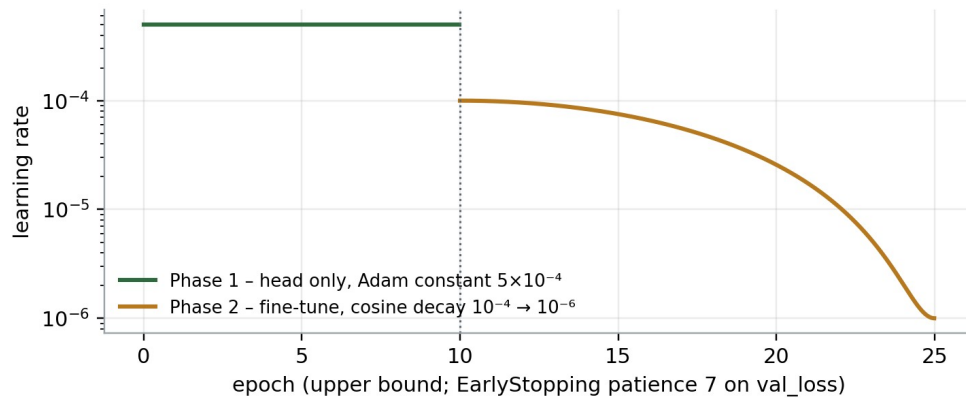
Augmentation. Standard geometric and photometric augmentation (rotation $\pm 40^\circ$, shifts 0.25, shear 0.2, zoom 0.3, brightness 0.6–1.4, channel shift 30, horizontal flip, reflective fill) was combined with a custom `field_augment` function that makes clean laboratory images resemble phone photographs (Table 15). Vertical flipping was disabled because leaves have a consistent tip-to-stalk orientation.

Table 15: Field-realism augmentation applied during training

Transform	Probability	Parameters	Real-world artefact simulated
Gaussian blur	0.35	kernel 3 or 5	Out-of-focus or motion-blurred photo
Additive Gaussian noise	0.30	σ uniformly in [4, 14]	Sensor noise in poor light
Directional shadow	0.30	linear ramp, strength 0.5–0.85	Sun angle; the photographer's shadow

Table 16: Training hyperparameters

Setting	Phase 1	Phase 2	Rationale
Trainable layers	Head only (748,084 params)	All except the first 100 backbone layers	Keep generic early edge and texture filters
Optimiser / learning rate	Adam, 5×10^{-4} (ReduceLROnPlateau $\times 0.5$, patience 3)	Adam, cosine decay $10^{-4} \rightarrow 10^{-6}$ ($\alpha = 0.01$)	Avoid destroying pre-trained features
Maximum epochs	10	15	Both limited by early stopping
Batch size	64	64	Throughput on a T4 GPU
Loss	Categorical cross-entropy, label smoothing 0.1	Same	Reduce over-confidence (Müller, Kornblith and Hinton, 2019)
Regularisation	Dropout 0.5 $\times 2$; L2 1×10^{-4}	Same	Small classes overfit quickly
Checkpoint	Best validation macro-F1	Best validation macro-F1	Equal weight for every class
Early stopping	val_loss, patience 7, restore best	Same	Stop at the validation optimum
Precision / seed	mixed_float16; seed 42	Same	Speed (Micikevicius <i>et al.</i> , 2018); reproducibility

**Figure 14:** Learning-rate schedule across the two training phases (upper bound on epochs)

Model iterations. The deployed model is the result of several iterations, summarised in Table 17. Early MobileNetV2 and EfficientNetB3 experiments informed the choice of backbone; successive EfficientNetB2 runs added the Unknown class, the field data and finally the open-set statistics. Only the weights, class list and centroid statistics produced by the same run are valid together, because centroids live in the feature space of one specific checkpoint; the backend therefore loads all three from one folder and checks that the weights and class list agree before starting.

Table 17: Model iterations retained in the repository

Folder	Date	Backbone / classes	Notable change
MobileNetV2-model, fyp_efficientnet_b3_model	Before 28 Jun 2026	MobileNetV2; EfficientNetB3 (300×300)	Baseline backbone experiments
best_model	27 Jul 2026	EfficientNetB2	First B2 model integrated with the API
Final_Model	31 Aug 2026	EfficientNetB2, 51 classes	Lab training run with saved curves and confusion matrix
final_field_Model	4 Sep 2026	EfficientNetB2, 52 classes	Field images and Unknown class; open-set threshold 0.6764
last_final_model (deployed)	6 Sep 2026	EfficientNetB2, 52 classes	Final run; 51 centroids × 512 dimensions, threshold 0.5596

6.1.3 Open-set calibration

After training, the notebook embeds the full training set through `dense_hidden`, L2-normalises the embeddings and computes one normalised centroid per known class; the Unknown class is excluded because it has no coherent centre. The threshold is set at the 5th percentile of the maximum centroid similarity over known validation images, so by construction about 5% of genuine known leaves are rejected in exchange for catching unfamiliar ones. For the deployed model this produced 51 centroids of 512 dimensions and $\tau = 0.5596$, saved as `ood_stats.npz`. At inference a dual-output model returns probabilities and the embedding in a single forward pass, so the check adds negligible cost. During integration a mis-named statistics file in an earlier model folder silently disabled this rule; the fix was to regenerate the file from the same run and to log at start-up whether open-set rejection is enabled.

6.1.4 Inference API

The FastAPI service rebuilds the exact training architecture and loads the weights file rather than a serialised model, which avoids Keras version incompatibilities. Key implementation decisions and the problems they solved were:

- **Leaf cropping with safety rails.** `crop_to_leaf` builds a mask from the excess-green index ($2G - R - B > 15$) plus a narrow diseased-tissue hue band that excludes dull soil, applies morphological opening and closing, and crops to the largest contour with 10% padding. If the region is below 5% or above 90% of the frame, or thinner than 15% of either dimension, the original image is used unchanged, so cropping can help but cannot destroy an image.
- **Preprocessing parity.** Images are resized to 224×224 and passed to EfficientNet's own `preprocess_input` on raw 0–255 values, identically in the notebook, the API and a standalone tester (`predict_test.py`); incorrect manual rescaling is a common silent cause of accuracy loss.
- **Grad-CAM under Keras 3.** Two silent failures were diagnosed and fixed: Keras 3 removed `Layer.output_shape`, so the search for the last convolutional layer failed and heatmaps were disabled without error; and a redundant `.numpy()` call on an array raised an exception that was swallowed. The layer search now inspects the

output tensor rank, and dependencies are pinned exactly so a rebuild cannot silently reintroduce the problem.

- **Robust error handling.** Empty uploads, undecodable files and decompression-bomb errors are mapped to HTTP 400 with actionable messages instead of HTTP 500; NumPy types are converted recursively so JSON encoding cannot fail; and a database connection failure at start-up is logged but does not stop prediction.
- **Two kinds of unknown.** Distinguishing “Crop Not Supported” (the leaf does not resemble any trained crop; the message lists the supported crops, generated from the class list) from “Not Identified” (take a clearer photo) gives the user an actionable next step in each case.

6.1.5 Web application

The web client is a React 19 single-page application with eleven routes and a not-found page. The sign-in and create-account screens render on their own, without the header, footer or navigation, since there is nothing to navigate to until a session exists (Figure 20). Every other route (home, diagnose, result, history and history detail, disease library and library detail, about and profile) sits behind a route guard, `RequireAuth`, which sends any visitor without a valid session to the sign-in screen. An authentication context restores the token from `localStorage` on load and validates it with `/auth/me`, discarding it if the server no longer accepts it. All network access passes through one API module whose base URL is injected at build time (`VITE_API_BASE_URL`) and which attaches the Bearer token, and a pure normalisation function converts the raw response into what the interface renders in the selected language. Engineering details include a reference-based guard against duplicate submissions, typed error classification (timeout, network, server, client) with plain-language messages, a 12 MB upload limit and 60-second timeout enforced before sending, and client-side thumbnails so results survive a page refresh. The server saves each check itself when `/predict` receives a valid token. The History page downloads the account history on every visit and mirrors it into `localStorage`: the account is the single source of truth and the browser copy is only a cache, which keeps the page usable when offline and prevents the same check from appearing twice under a local and a server identifier. A header account menu shows the username and email and links to the profile, which shows the number of saved checks and offers sign-out (Figures 21 and 22). An earlier version (15 September) had no accounts and kept history only in the browser; accounts were made compulsory on 23 September so that a farmer's history follows them between phone and browser. Figures 15 to 19 show the implemented interface running against the real API.

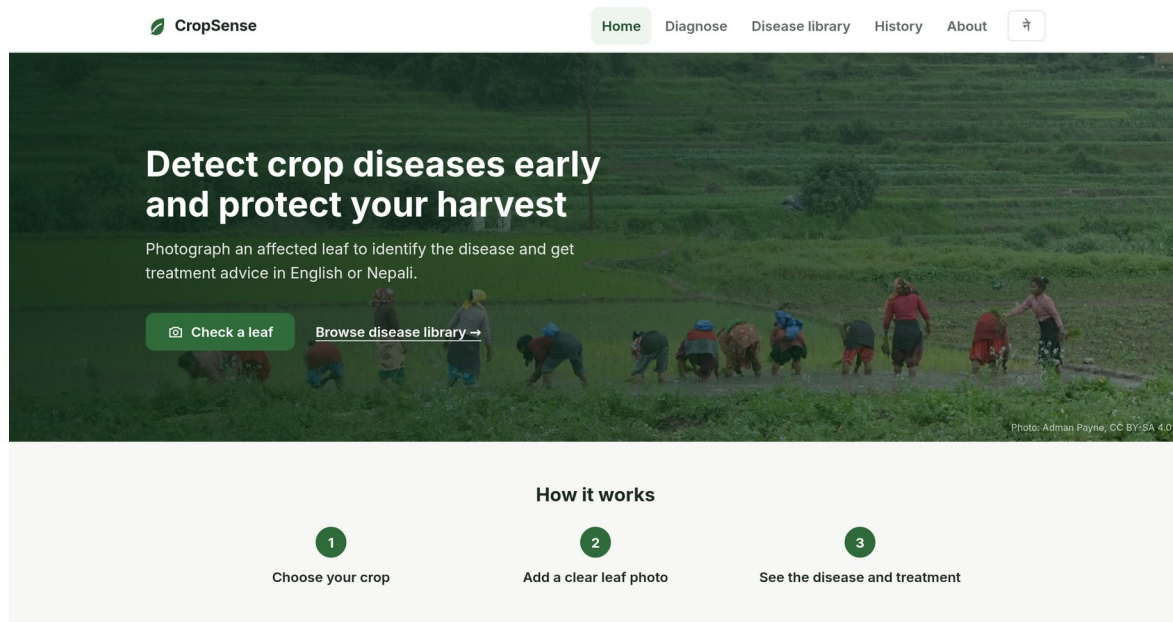


Figure 15: Home page with a Nepali farming photograph (Payne, 2018; CC BY-SA 4.0) and a single primary action

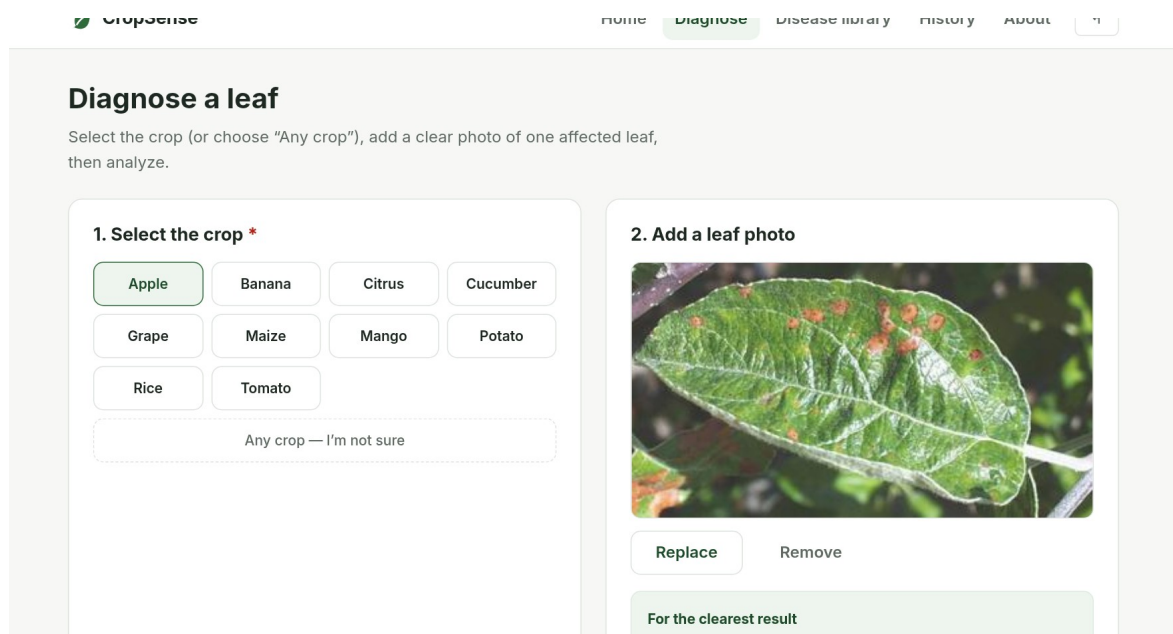


Figure 16: Diagnose page: crop selection, uploaded photo preview and photo-quality tips

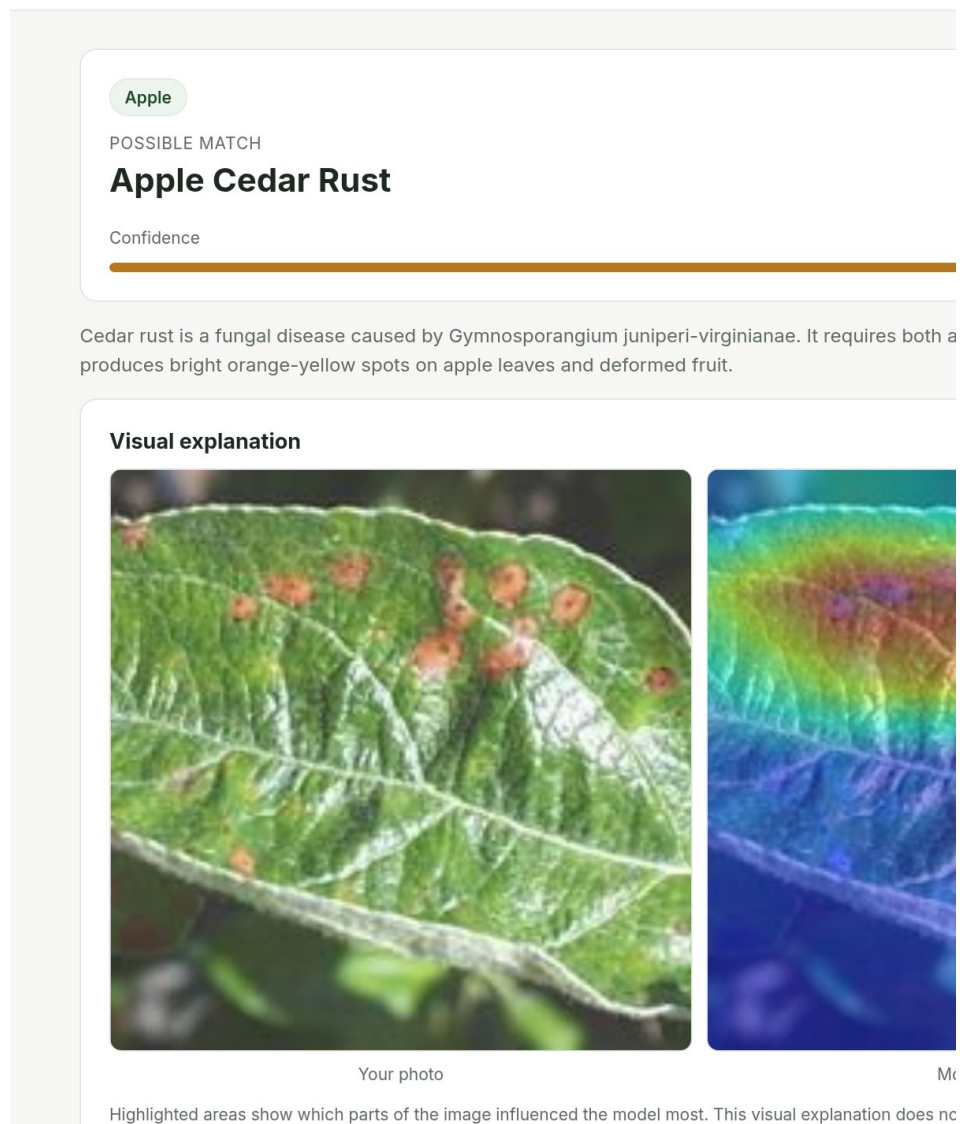


Figure 17: Result page for a PlantDoc apple leaf: possible match, confidence band and Grad-CAM comparison

Symptoms

Bright orange-yellow spots on upper leaf surfaces, horn-like tubes on leaf undersides, deformed fruit.

Cause

Fungal disease caused by Gy (requires cedar alternate host)

General management

Apply fungicide at pink bud stage. Remove nearby cedar trees if possible.

Prevention

Remove juniper/cedar alterna varieties.

Recommended treatment

Try the non-chemical step first — it costs nothing and leaves no residue. Take this page to your agrovet and not a brand.



Cultural control

Cultural Practice

Remove nearby juniper/cedar trees where possible - the fungus needs both hosts to complete its cycle.



Mancozeb 75% WP

Fungicide · **Slightly Hazardous**

Dose	2 g per litre of water
Formulation	Wettable powder — mix with water
Wait before harvest	14 days
Repeat	every 10 days if symptoms continue
Safety	Gloves and a mask are still recommended.



Buying it at the agrovet

Ask for the active ingredient by name: Mancozeb 75% WP. Brands and packaging differ, so check code WP, and that its hazard triangle matches the colour above.

Sold as a powder in a sealed sachet or packet.

Protectant only - it prevents new infection but cannot cure infected tissue. Spray before or at first symptom

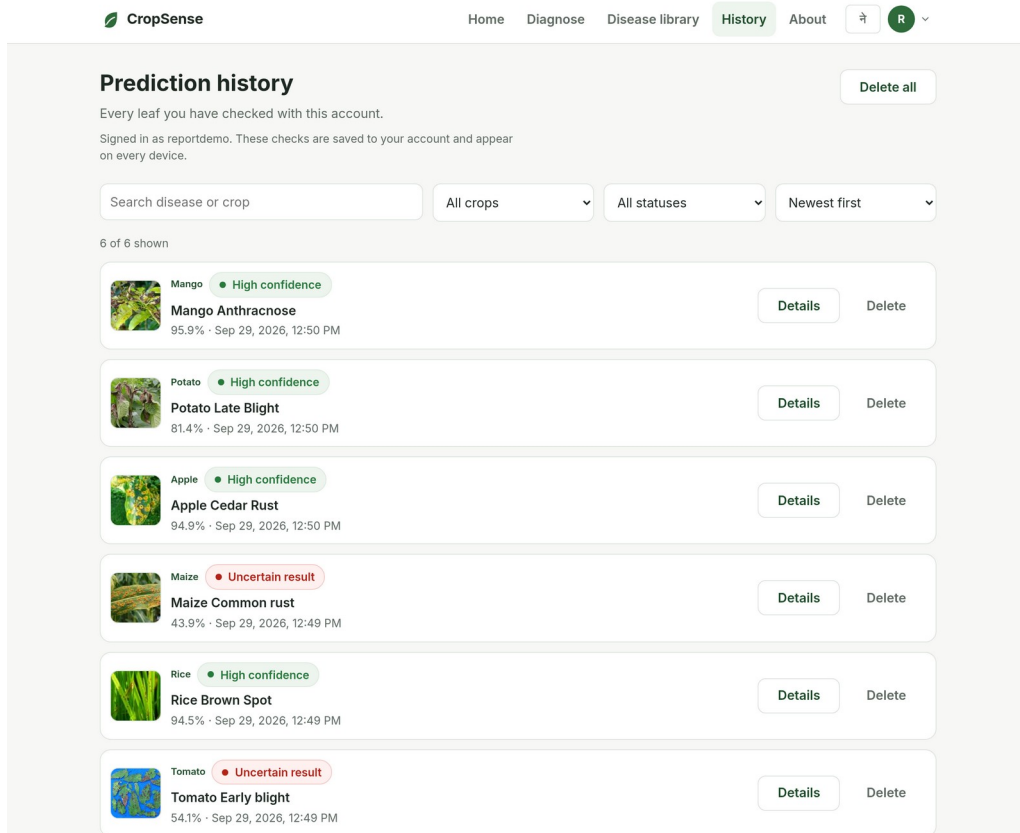


Myclobutanil 10% WP

Fungicide · **Moderately Hazardous**

Dose	1 g per litre of water
Formulation	Wettable powder — mix with water
Wait before harvest	21 days
Repeat	every 14 days if symptoms continue

Figure 18: Result page (continued): symptoms, cause, management, prevention and structured treatment options with hazard bands and disclaimer



CropSense Home Diagnose Disease library **History** About 🔍 R

Prediction history

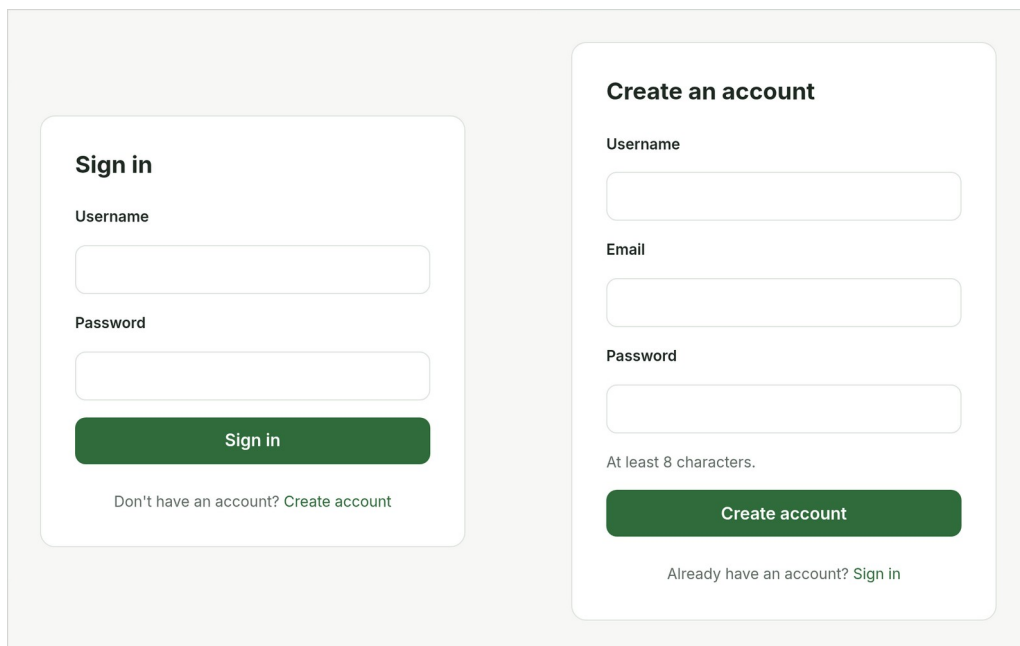
Every leaf you have checked with this account.
Signed in as reportdemo. These checks are saved to your account and appear on every device.

Search disease or crop All crops All statuses Newest first

6 of 6 shown

	Mango High confidence	Mango Anthracnose	95.9% · Sep 29, 2026, 12:50 PM	Details	Delete
	Potato High confidence	Potato Late Blight	81.4% · Sep 29, 2026, 12:50 PM	Details	Delete
	Apple High confidence	Apple Cedar Rust	94.9% · Sep 29, 2026, 12:50 PM	Details	Delete
	Maize Uncertain result	Maize Common rust	43.9% · Sep 29, 2026, 12:49 PM	Details	Delete
	Rice High confidence	Rice Brown Spot	94.5% · Sep 29, 2026, 12:49 PM	Details	Delete
	Tomato Uncertain result	Tomato Early blight	54.1% · Sep 29, 2026, 12:49 PM	Details	Delete

Figure 19: Account prediction history with thumbnails, confidence status, search, filters and deletion



Sign in

Username

Password

Sign in

Don't have an account? [Create account](#)

Create an account

Username

Email

Password

At least 8 characters.

Create account

Already have an account? [Sign in](#)

Figure 20: Sign-in and create-account screens, shown without header or footer until a session exists

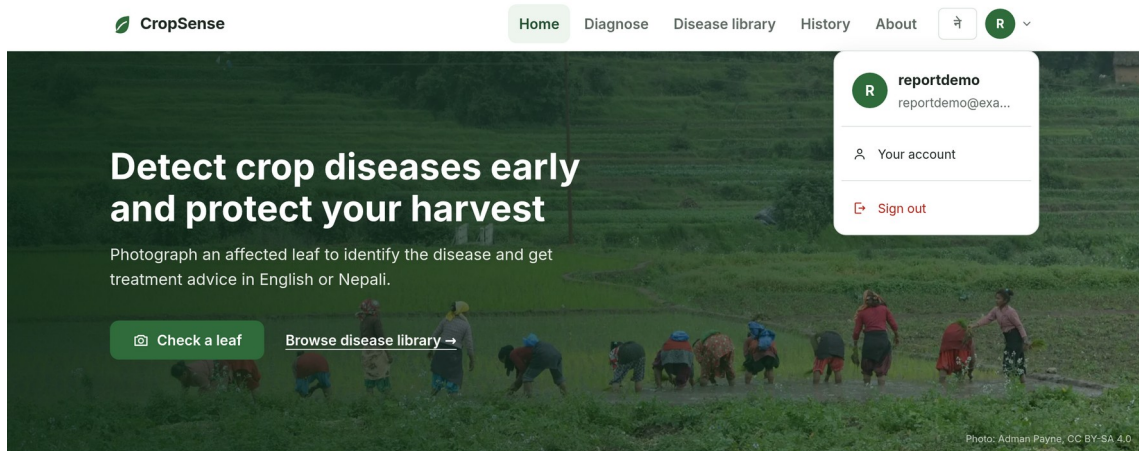


Figure 21: Account menu in the header: username, email, profile link and sign-out

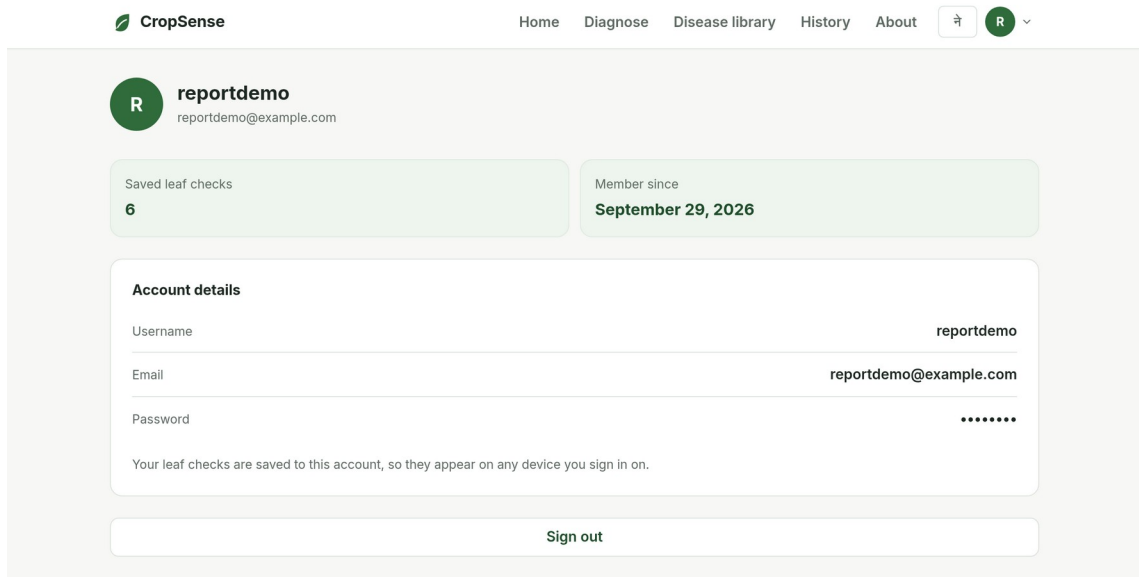


Figure 22: Profile page: number of saved checks, member-since date, account details and sign-out

The Nepali interface and the phone-width layout are shown in Appendix C.

6.1.6 Mobile application

The Android application is a thin Expo/React Native client (mobile/, about 2,100 lines of JavaScript in 16 files). On launch it restores a saved session from expo-secure-store, which keeps the token in the Android Keystore rather than plain storage, and checks it with /auth/me; without a valid session it shows a combined sign-in and create-account screen (AuthScreen.js). Once signed in, a slide-in sidebar (Sidebar.js) leads to the diagnosis screen, the account history (HistoryScreen.js, which uses the same /auth/history endpoints as the web client, so checks made on either appear on both) and the profile (ProfileScreen.js). The app requests camera or gallery permission at the point of use, captures photos at quality 0.8 through expo-image-picker, offers the same optional crop selection and duplicate-submit guard as the web client, and renders the same bilingual result, Grad-CAM image and treatment cards. Requests time out after 40

seconds for a prediction and 20 seconds for account calls. An English/Nepali toggle re-renders an existing result without a new request. The API address defaults to the public Hugging Face Space in code, because cloud builds ignore uncommitted environment files, and can be overridden with `EXPO_PUBLIC_API_URL` for local testing. On 29 September 2026 expo-doctor passed 20 of 21 checks: the failing check reports that three Expo packages are one patch release behind the latest for SDK 57, which is resolved by `npx expo install --check` and does not affect behaviour. The Android bundle exports without errors (717 modules), and a standalone APK (`cropsense-ai.apk`, 75 MB, package `com.nifn.cropsenseai`) was built with Expo EAS Build on 24 September 2026, so the app depends on neither Expo Go nor a development server.

6.1.7 Deployment

Figure 23 shows the deployment pipeline and Table 18 the resulting services. The GitHub repository holds the source code; a push to main automatically rebuilds the web client on Vercel. The backend and the 102 MB weights file (stored with Git LFS) are pushed to a Hugging Face Space repository, which installs the pinned requirements and serves the API over HTTPS. Accounts and history are stored in a Neon serverless PostgreSQL database (free tier, Singapore region) reached over TLS. Its connection string (`DATABASE_URL`) and the token-signing key (`JWT_SECRET`) are held as secrets in the Space settings and are never committed.

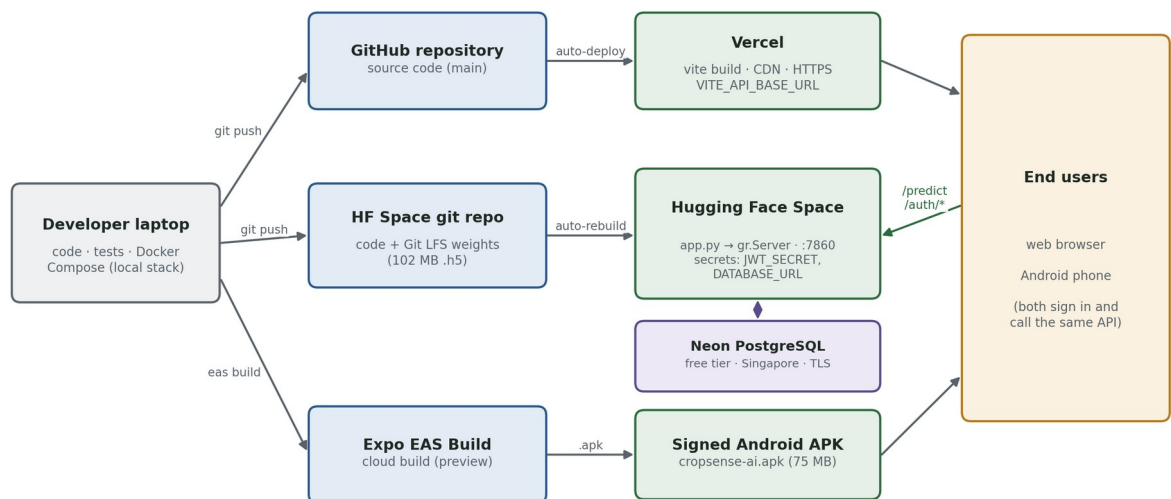


Figure 23: Deployment pipeline for the web client, API and Android application

Table 18: Deployed services

Service	Platform	Address / artefact	Notes
Inference API	Hugging Face Space (Gradio SDK 6.27, CPU)	https://bikkii-cropsense.hf.space	Sleeps when idle; wakes on request
Web client	Vercel (Vite static build, CDN)	Vercel project URL	Auto-deploys on push; SPA rewrite rule

Service	Platform	Address / artefact	Notes
Database	Neon serverless PostgreSQL 18 (free tier, Singapore)	DATABASE_URL secret on the Space	Sleeps when idle; first connection wakes it; TLS required
Android client	Expo EAS Build	Signed .apk (preview profile)	Installed directly on devices
Local full stack	Docker Compose	localhost:3000 (web), :8000 (API)	Offline demonstration fallback

Deployment raised four problems worth recording. First, Hugging Face had moved Docker-based Spaces to a paid tier, so the service was adapted to the free Gradio SDK. Second, the first Gradio deployment failed with “address already in use” because the API was started with its own server rather than through Gradio's launcher; it was fixed by serving the FastAPI routes through Gradio's `gr.Server`, which starts through Gradio's own launch function. Third, the first Vercel build failed because a Python template rule (`lib/`) in `.gitignore` had silently excluded the web client's `src/lib` folder from version control; the rule was anchored to the repository root and the fix verified by building and testing from a fresh clone.

Fourth, when the hosted database was connected, every TLS connection to Neon crashed the whole backend process at connect time with no traceback. The cause was that `psycpg2` and TensorFlow each bundle their own OpenSSL library, and whichever loads first wins; importing `psycpg2` before TensorFlow fixed it, and a comment in `main.py` records why the import order must not change.

Table 19: Key construction challenges and decisions

Challenge	Decision / resolution	Consequence
Severe class imbalance (104:1)	Gentle three-layer balancing; macro-F1 checkpointing	Rare classes not ignored by checkpoint selection
Lab-to-field gap	Field data, field augmentation, leaf cropping	Gap reduced but still large (Section 6.2.4)
Untrained crops forced onto known classes	Unknown class + softmax rules + centroid distance	Mechanism works end to end; weak separation on PlantDoc
Silent Grad-CAM failures after Keras 3	Rank-based layer search; exact dependency pins	Heatmap produced for 226/226 evaluated images
Colab disconnections	Per-epoch Drive checkpoints and resume logic	Long runs completed on free GPUs
Free hosting memory and payment constraints	EfficientNetB2; Hugging Face Gradio Space; Vercel	£0 public deployment
Farmers' language and trust	Bilingual knowledge base; confidence bands; disclaimers; free account with history on every device	Accessible, cautious presentation
History scattered across devices; no owner for saved data	Compulsory accounts; server-side history; browser copy used only as a cache	Same history on web and Android; duplicates avoided
TLS connection to hosted PostgreSQL crashed the process	Import <code>psycpg2</code> before TensorFlow (conflicting OpenSSL builds)	Stable connection to Neon
Account history too large for phones (~400 KB heatmap per row)	List returns thumbnails only; heatmap fetched per entry	7.9 KB list response for one entry; no phone timeouts
Times shown 5 h 45 min early in Nepal	Label naive database timestamps as UTC before sending	Correct local times on every client

Challenge	Decision / resolution	Consequence
Public repository could expose a signing key	No hard-coded JWT fallback; random per-process key with a warning if unset	A misconfigured deployment fails closed

Together these components meet the problem statement in its practical dimension: a Nepali farmer can now photograph a leaf on a phone or in a browser, at no cost with a free account, and receive a cautious, explained, bilingual answer with safety-conscious guidance. Whether that answer is accurate enough in the field is the subject of the next section.

6.2 Validation and Testing

Validation addressed four questions: does the software behave correctly (functional testing), does it handle invalid and adversarial input safely (robustness), how accurate is the model on real photographs (model evaluation), and is it fast enough (performance). Table 20 summarises the strategy.

Table 20: Validation and testing strategy

Level	Technique	Tooling	Result
Unit / component	Automated tests of normalisation, history store, image picker, result view, history page and sign-in flow	Vitest 3.2, React Testing Library, jsdom	28/28 passed
Static analysis & build	Lint; production build; Expo project checks; Android bundle export	ESLint, Vite, expo-doctor, expo export	Clean; builds succeed; expo-doctor 20/21 (3 packages one patch behind)
API black-box	Scripted edge cases and crop-mismatch checks against the running API	Python requests	8/9 as specified; 1 defect
Auth & history API	Scripted sign-up, sign-in, token and ownership tests against the local stack	Python requests, Docker Compose	17/17 as specified
Model (internal)	Training/validation curves and confusion matrix from a laboratory run	Notebook outputs	Val. accuracy 98.5–98.9% (lab)
Model (independent)	Deployed endpoint on PlantDoc field photos and untrained crops	Evaluation script	Section 6.2.4
Performance	Latency locally and on the live Space	Timed HTTP requests	0.79 s local; 6.75 s live
Deployment smoke test	Health, prediction, CORS and page routing on live services	curl, browser	Passed

6.2.1 Automated software tests

The frontend suite concentrates on the logic most likely to cause harmful mistakes rather than chasing coverage percentages. All 28 tests pass and ESLint reports no problems (Table 21).

Table 21: Automated frontend test suite

Test file	Tests	What it protects
lib/normalize.test.js	10	Mapping of raw responses to High/Moderate/Uncertain, doubt flags forcing Uncertain, language selection, unknown and not-leaf outcomes
lib/history.test.js	4	Saving, ordering, removing and clearing local history records
components/ImagePicker.test.jsx	3	Photo selection, preview and guidance behaviour

Test file	Tests	What it protects
components/ResultView.test.jsx	4	Responsible wording, Grad-CAM caveat, uncertainty warning, feedback controls
pages/History.test.jsx	3	Delete-all control visibility, confirmation and clearing
pages/Auth.test.jsx	4	A signed-out visitor is sent to a bare sign-in screen; signing in stores the token and shows the username; signing out clears the token; wrong credentials show an error
Total	28	All passed (Vitest 3.2.7 run, 29 September 2026)

6.2.2 Internal training evidence

The training notebook computes a full classification report, a field-only score and rejection statistics, but the cell outputs of the run that produced the deployed model were not saved, so those numbers cannot be reported. The only preserved training artefacts come from the 31 August laboratory run (`Final_Model`, 51 classes), shown in Figures 24 and 25. They are reported for what they are and are not attributed to the deployed model.

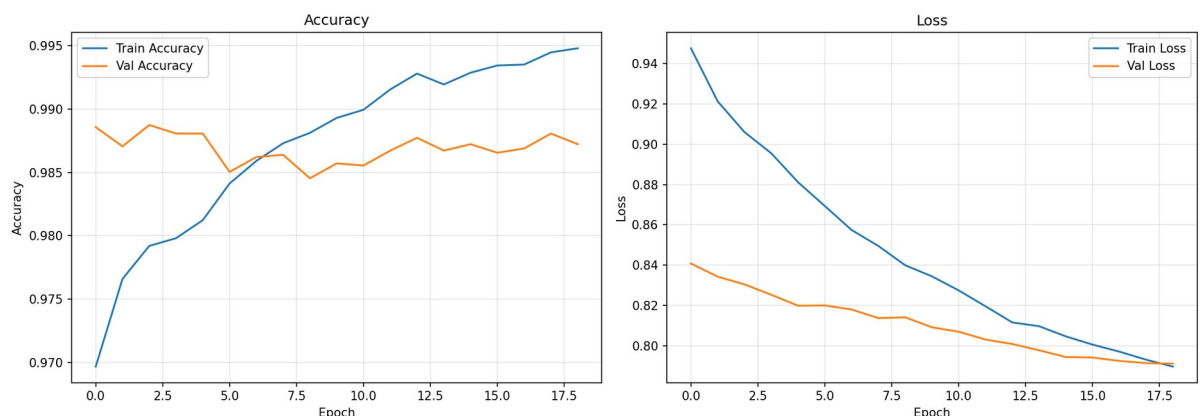


Figure 24: Training and validation accuracy and loss over 19 epochs for the 31 August 2026 laboratory run

Validation accuracy stayed within a narrow band of about 0.985–0.989 while training accuracy rose from 0.970 to 0.995, crossing validation accuracy around epoch 6. Validation accuracy starts above training accuracy because training metrics are computed with dropout and augmentation active. Losses plateau near 0.79 rather than approaching zero because label smoothing places a floor under cross-entropy. The widening gap after epoch 6 indicates mild overfitting, guarded against by early stopping and macro-F1 checkpointing. The confusion matrix is strongly diagonal. Both results are consistent with the literature's near-saturated laboratory accuracy and, as Section 2.0.3 predicts, say little about field performance.

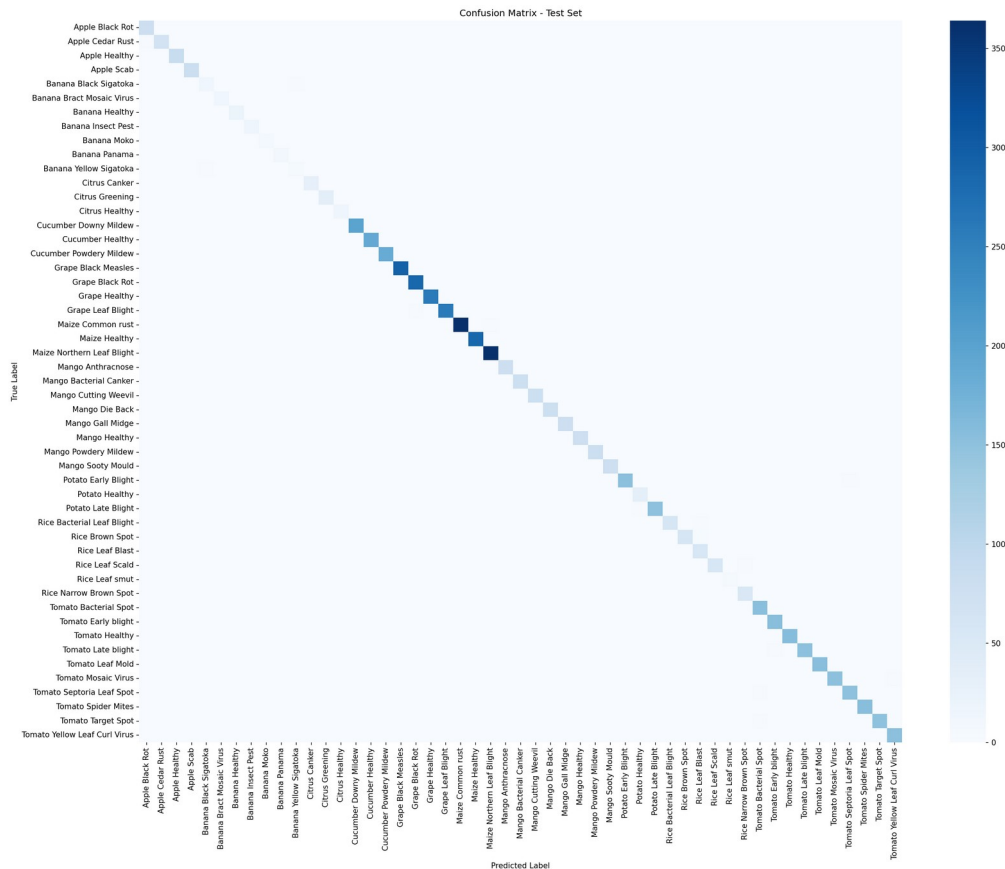


Figure 25: Laboratory test-set confusion matrix from the 31 August 2026 run (51 classes, before the Unknown class was added)

6.2.3 API robustness and functional tests

Table 22 lists black-box tests executed against the running API. Eight of nine behaved as specified. The ninth exposed a defect: the code sets Pillow's `MAX_IMAGE_PIXELS` to 120 million intending to reject larger images, but Pillow only raises `DecompressionBombError` above twice that limit and merely warns between one and two times it, so a 132-megapixel image was fully decoded rather than rejected. The pipeline still returned a safe result ("Not a Leaf"), but the memory protection is weaker than intended. The fix is to check the decoded dimensions explicitly and return HTTP 400 above the limit; it is recorded as future work. A further observation is that re-saving the same apple-rust photograph as a JPEG at Pillow's default quality changed the model's confidence from 85.4% to 77.4% with the same predicted class, showing sensitivity to compression that phone cameras and messaging applications routinely introduce.

Table 22: API black-box and edge-case test results

Test case	Expected	Observed (HTTP status; outcome)	Verdict
Empty file	400, no image uploaded	400; No image was uploaded.	Pass
Text file renamed	400, unreadable image	400; That file could not be read as an image. Please upload a JPG, PNG or WEBP photo.	Pass
Decompression bomb 132 MP	400, image too large	200; Not a Leaf; unknown=True not_leaf=True mismatch=False	Defect

Test case	Expected	Observed (HTTP status; outcome)	Verdict
Blank white image	Not a Leaf	200; Not a Leaf; unknown=True not_leaf=True mismatch=False	Pass
Random RGB noise	Not a Leaf or unknown	200; Crop Not Supported (94.58%); unknown=True not_leaf=False mismatch=False	Pass
Text document image	Not a Leaf	200; Not a Leaf; unknown=True not_leaf=True mismatch=False	Pass
Wide farm landscape (no single leaf)	Not a Leaf or unknown	200; Crop Not Supported (86.14%); unknown=True not_leaf=False mismatch=False	Pass
Rice leaf (field_data), crop=Rice	Correct disease, no warning	200; Rice Bacterial Leaf Blight (94.93%); unknown=False not_leaf=False mismatch=False	Pass
Same rice leaf, crop=Tomato (mismatch)	Same disease, mismatch warning	200; Rice Bacterial Leaf Blight (94.93%); unknown=False not_leaf=False mismatch=True	Pass

Authentication and account history. The account endpoints were tested black-box on 29 September 2026 against the full local stack (FastAPI backend and PostgreSQL 16 in Docker Compose), using a scripted test with two throw-away accounts. All 17 cases behaved as specified (Table 23). Two results matter most. A request to /predict with a tampered token still returned a diagnosis but did not save it, confirming that session problems cannot block the core function. A second user could neither read nor delete the first user's saved check. One further observation is a weakness rather than a failure: twenty consecutive wrong passwords for the same account all returned HTTP 401 with no delay or lockout, so login is not protected against online guessing (Section 8.1).

Table 23: Authentication and account-history API test results

Test case	Expected	Observed (HTTP status; detail)
Sign-up, 2-character username	400	400; Username must be at least 3 characters.
Sign-up, malformed email	400	400; Please enter a valid email address.
Sign-up, 7-character password	400	400; Password must be at least 8 characters.
Sign-up, valid	200	200; User created successfully. Please log in.
Sign-up, duplicate username	400	400; Username already exists
Sign-in, wrong password	401	401; Invalid username or password
Sign-in, valid	200 + token	200; JWT (HS256) with username and email
GET /auth/me without token	401 or 403	401; Not authenticated
GET /auth/me with tampered token	401	401; Invalid or expired token
GET /auth/me with valid token	200	200; username, email, created_at (UTC)
POST /predict with tampered token	200, not saved	200; Tomato Early blight; history unchanged
POST /predict signed in	200, saved	200; Tomato Early blight; one history row
GET /auth/history	200, 1 entry	200; 1 entry, thumbnail only (7.9 KB response)
Second user reads that entry	403	403; Not authorized to view this prediction
Second user deletes that entry	403	403; Not authorized to delete this prediction
Owner reads full entry	200 + heatmap	200; heatmap of 405 KB base64 included
Unknown entry id	404	404; Prediction not found

6.2.4 Independent field evaluation on PlantDoc

Protocol. The deployed model was evaluated through the real /predict endpoint, including leaf cropping, the leaf pre-check and the unknown gate, on the complete PlantDoc test split (236 images, 27 folders; Singh *et al.*, 2020). PlantDoc was collected independently of this project, but because both it and the CropSense field set were sourced from web image search, every PlantDoc image was compared with all 1,593

CropSense field images using average and difference hashes; 10 near-duplicates were removed to prevent leakage, leaving 226 images. Folders were mapped to CropSense classes (Table 24): 17 folders (148 images) correspond to supported classes; 9 folders (74 images) are crops the model was never trained on (bell pepper, blueberry, cherry, peach, raspberry, soybean, squash and strawberry); and corn grey leaf spot (4 images) is a supported crop with an unsupported disease. No crop was specified, matching the “Any crop” option. Both model-level metrics (the raw top-1 and top-5 classes, ignoring the gate) and system-level outcomes (what the user is shown) were recorded.

Table 24: Mapping of the PlantDoc test split to evaluation groups

Group	PlantDoc folders (mapped CropSense class)	Images
Supported classes	Apple leaf (Apple Healthy); Apple rust leaf (Apple Cedar Rust); Apple Scab Leaf; Corn leaf blight (Maize Northern Leaf Blight); Corn rust leaf (Maize Common rust); grape leaf (Grape Healthy); grape leaf black rot; Potato leaf early blight; Potato leaf late blight; Tomato Early blight leaf; Tomato leaf (Tomato Healthy); Tomato leaf bacterial spot; Tomato leaf late blight; Tomato leaf mosaic virus; Tomato leaf yellow virus (Yellow Leaf Curl Virus); Tomato mold leaf (Leaf Mold); Tomato Septoria leaf spot	148
Untrained crops	Bell_pepper leaf; Bell_pepper leaf spot; Blueberry leaf; Cherry leaf; Peach leaf; Raspberry leaf; Soyabean leaf; Squash Powdery mildew leaf; Strawberry leaf	74
Unsupported disease	Corn Gray leaf spot	4
Removed	Near-duplicates of CropSense field training images	10

Table 25: Independent field evaluation results for supported classes

Metric (supported classes, n = 148)	Value
Top-1 accuracy (model)	48.0% (71/148)
Top-5 accuracy (model)	81.8% (121/148)
Correct crop in top-1	70.9% (105/148)
Images rejected by the leaf pre-check	0% (0/148)
System: result shown and correct	43.2% (64/148)
System: result shown but wrong	33.8% (50/148)
System: declined (17 Crop Not Supported, 17 Not Identified)	23.0% (34/148)
Precision of shown results	56.1% (64/114)
Accuracy when confidence $\geq$ 80% (coverage)	81.8% on 22.3% of images
Accuracy when confidence $\geq$ 60% (coverage)	72.3% on 43.9% of images

Accuracy. On independent field photographs of supported classes the model's top-1 accuracy was 48.0%, far below the 98–99% laboratory validation accuracy, while top-5 accuracy was 81.8% and the crop was identified correctly 70.9% of the time. The size of this drop closely mirrors that reported by Mohanty, Hughes and Salathé (2016) and confirms that the domain gap, although narrowed by field data and augmentation, remains the dominant limitation. Performance varied widely between classes (Figure 26): maize common rust (89%), grape healthy (83%), apple healthy, maize northern leaf blight and tomato early blight (67%) performed well, whereas tomato bacterial spot (0/8), tomato late blight (1/9) and tomato healthy (1/8) performed poorly. Per-class samples are small (5–12 images), so these per-class figures carry wide uncertainty.

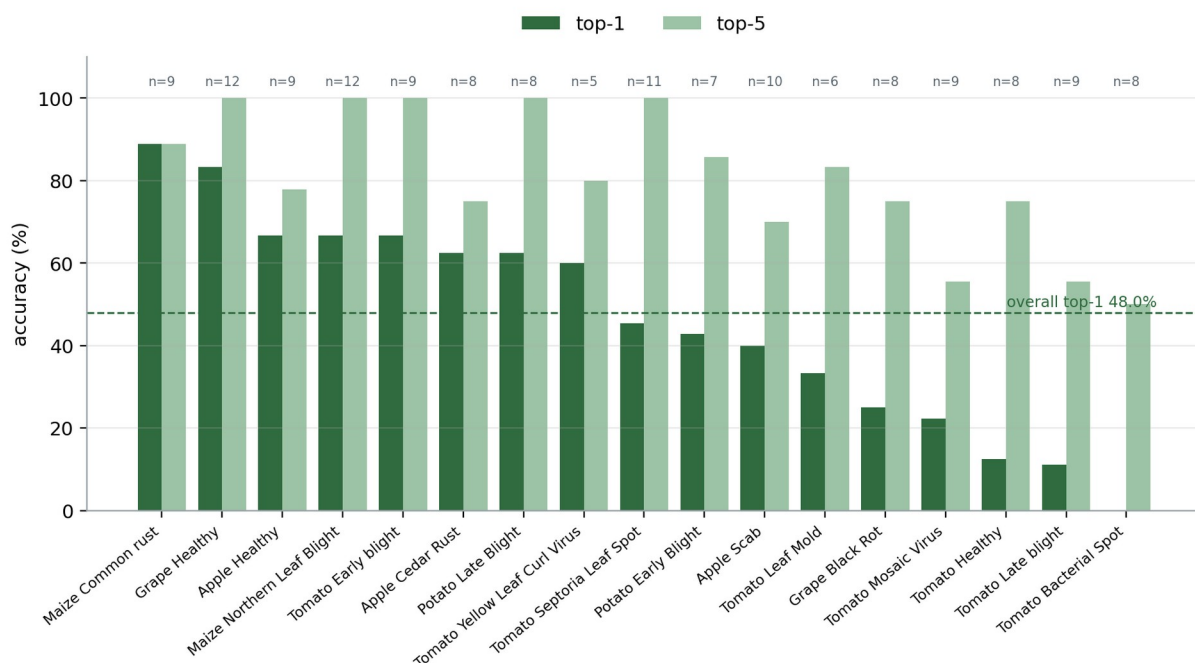


Figure 26: Per-class top-1 and top-5 accuracy on PlantDoc field photographs (supported classes)

The errors are largely agronomically plausible rather than random. The most frequent confusions were tomato Septoria leaf spot predicted as tomato early blight (4 cases), maize northern leaf blight as maize common rust (3), tomato late blight as potato late blight (3), grape black rot as cucumber downy mildew (3) and tomato healthy as potato or cucumber healthy (2 each). Several pairs share a pathogen family or leaf morphology: tomato and potato are both Solanaceae and suffer the same *Phytophthora infestans* late blight. This suggests that crop selection by the user, which currently only triggers a mismatch warning, could usefully constrain the prediction.

Outcomes shown to users. Figure 27 shows what users would actually see. For supported crops the system gave a correct answer for 43.2% of photographs, declined 23.0% and showed a wrong answer for 33.8%. When the system did show an answer it was correct 56.1% of the time. Confidence is informative: Figure 28 shows that restricting answers to the most confident predictions sharply raises accuracy, to 81.8% for predictions of at least 80% confidence (22.3% of photographs) and 72.3% at 60% or more (43.9%). The current High band threshold (80%) is therefore meaningful, but the Uncertain band covers most field photographs.

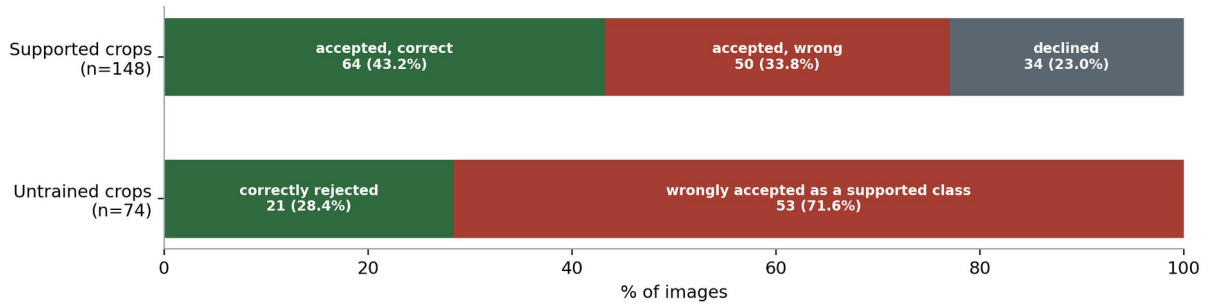


Figure 27: System-level outcomes on PlantDoc for supported and untrained crops

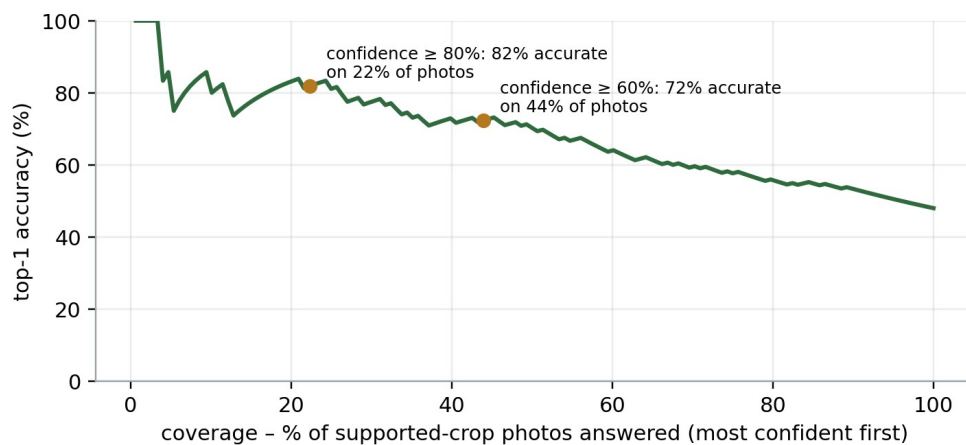


Figure 28: Accuracy versus coverage when answering only the most confident predictions (selective prediction)

Rejection of untrained crops. This is the weakest result. Of 74 leaves from untrained crops, only 21 (28.4%) were declined: 6 as Crop Not Supported and 15 as Not Identified. The remaining 53 were shown as a supported class, most often as a healthy class (apple healthy 12, mango healthy 8, grape healthy 8, potato healthy 4); squash powdery mildew was labelled cucumber powdery mildew, which is biologically reasonable. Rejection varied by species, from 7 of 11 blueberry leaves to none of 8 strawberry or 6 squash leaves. Figure 29 explains why: the distributions of the open-set score overlap almost completely (mean 0.701 for supported and 0.711 for untrained crops), so the cosine threshold rejected 11.5% of supported leaves but only 8.1% of untrained ones. On this benchmark the feature-distance rule provided no useful separation, and most successful rejections came from the softmax uncertainty rules. The 5% false-rejection rate targeted at calibration also rose to 11.5% on field images, showing that a threshold calibrated on in-domain validation data does not transfer across the domain gap.

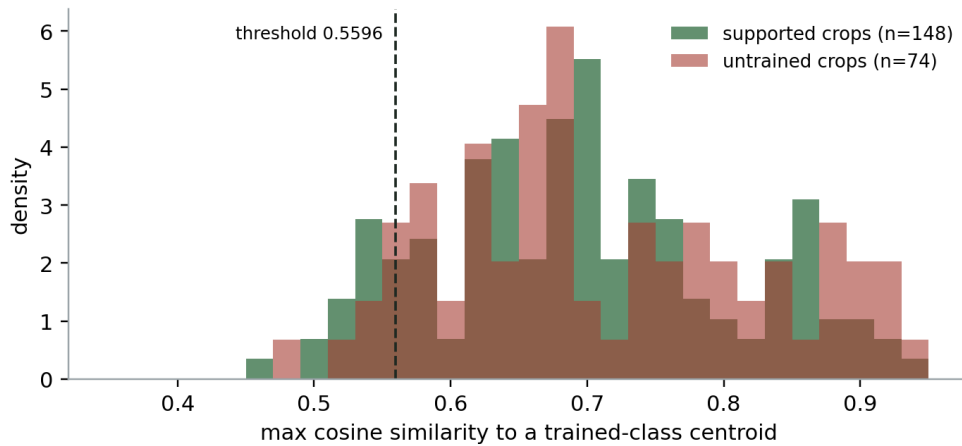


Figure 29: Distribution of the open-set score for supported and untrained crops on PlantDoc, with the calibrated threshold

Explanations. A Grad-CAM heatmap was generated for all 226 evaluated images. Figure 30 shows representative examples. For correct predictions the heatmap generally covers the lesions (maize rust pustules, apple rust spots, grape black-rot lesions). The fourth example is the instructive one: a tomato *Septoria* leaf spot image confidently (88%) predicted as tomato early blight still produces a plausible heatmap over genuine lesions. This confirms, in the artefact itself, the caution from Adebayo *et al.* (2018) that an explanation shows where the model looked, not whether it was right, and it validates the decision to label heatmaps accordingly in the interface.

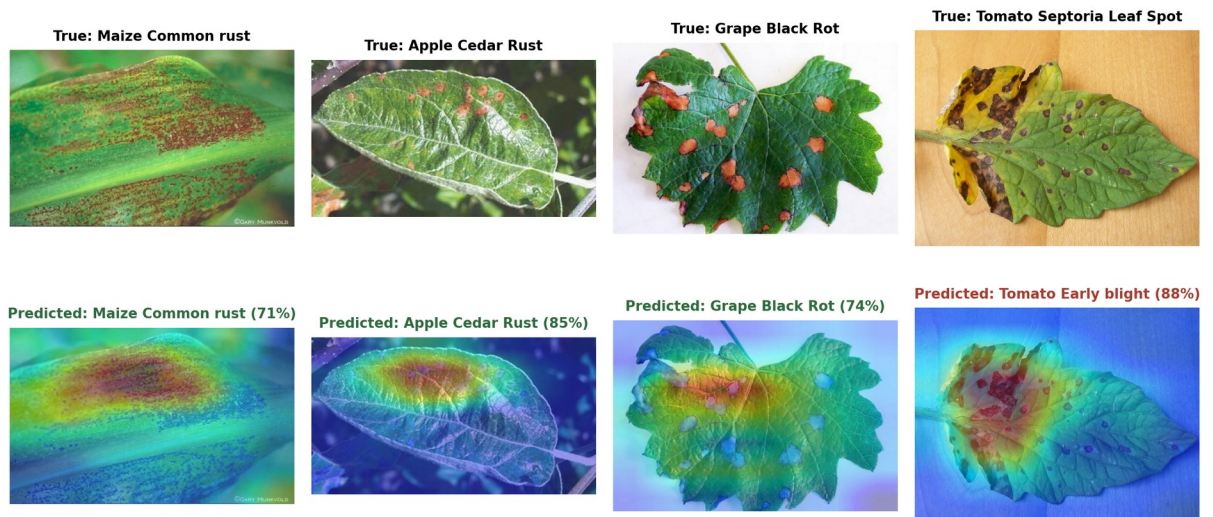


Figure 30: Grad-CAM explanations for PlantDoc photographs: three correct predictions and one confident misclassification

Validity. Several factors limit these conclusions. PlantDoc labels are themselves web-sourced and noisy, and some images are not simple leaf photographs, which depresses measured accuracy; the evaluation covers only 17 of the 51 classes and none of the banana, citrus, cucumber, mango or rice classes most relevant to Nepal; and the sample of 226 images yields wide confidence intervals (for example, the 95% Wilson interval for

48.0% top-1 accuracy on 148 images is roughly 40%–56%). The results should therefore be read as strong evidence of a large domain gap and weak open-set separation, rather than as precise estimates.

6.2.5 Performance

Response time was measured end to end, including upload, preprocessing, inference, Grad-CAM and JSON encoding (Figure 31). On the development laptop, sequential requests averaged 0.79 s; under three concurrent clients during the PlantDoc run the median was 2.18 s (90th percentile 3.26 s). On the live Hugging Face Space, warm sequential requests averaged 6.75 s ($n = 6$), within the 10 s target but noticeably slower because of shared CPU resources. A Space that has gone to sleep takes considerably longer for the first request while the model loads. Grad-CAM accounts for most of the computation. Saving the result for a signed-in user adds little: in the local stack with PostgreSQL, five sequential signed-in predictions took 0.75–0.87 s each including the database insert.

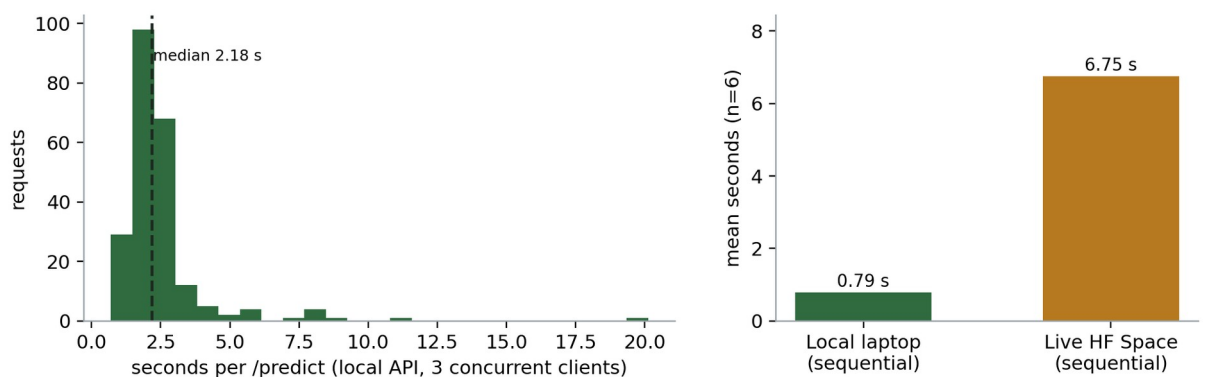


Figure 31: End-to-end /predict latency: distribution during the local evaluation and mean sequential latency locally and on the live Space

Deployment verification. The live API returned `model_loaded: true` with 52 classes (last checked on 29 September 2026), served correct predictions with CORS headers for a Vercel origin, and the web client built and passed its tests from a fresh clone. These smoke tests confirm that the evaluated behaviour is the deployed behaviour.

7. Critical Evaluation

CropSense AI is a working, publicly accessible decision-support system: a user photographs a crop leaf and receives a possible disease match with a confidence band, a visual explanation and bilingual, safety-conscious guidance, on the web or on an Android phone, at no cost, with a free account that keeps a history of every check across devices. This section judges how far it achieves its aim, how it compares with existing work and what it contributes.

7.1 Evaluation Against Objectives and Existing Solutions

Table 26: Assessment of the project against its SMART objectives

Objective	Evidence	Assessment
O1 Literature review and gap	50 sources cited; gap in field evaluation, unknown-crop handling, qualified explanation and local-language delivery (Section 2)	Achieved
O2 Dataset	10 crops, 51 classes; 38,348 lab + 1,593 cleaned field images; Unknown class; 274 duplicates and 43 collisions logged	Achieved ; field labels not expert-verified
O3 Model	EfficientNetB2, 8.52 M parameters; balancing, field augmentation, macro-F1 checkpointing; open-set statistics; lab validation accuracy ~98.7%	Largely achieved ; held-out metrics for the deployed run not preserved
O4 System	API with validation, leaf check, unknown gate, Grad-CAM; accounts and server-side history; web and Android clients; EN/NP content; 122 treatment entries	Achieved
O5 Deployment and testing	Live Hugging Face Space, Vercel and Neon database; APK build; 28/28 automated tests; 17/17 auth API tests; 8/9 edge tests with one defect found	Achieved ; one defect outstanding
O6 Evaluation	Independent PlantDoc evaluation: 48.0% top-1, 81.8% top-5, 28.4% untrained-crop rejection; latency 0.79 s local, 6.75 s live	Achieved as an evaluation; results show model performance is insufficient for unsupervised field use

Strengths. The engineering objectives were met in full. The artefact is complete rather than a notebook: a single inference service shared by two clients, bilingual content that includes treatment safety information, and deliberate handling of the three outcomes a naive classifier never produces (not a leaf, not identified, crop not supported). Presentation choices, such as possible-match wording, confidence bands that any doubt flag downgrades, and explicit caveats on heatmaps, embody responsible design. The system is free to use and runs on free infrastructure. Practical robustness is good: no evaluated field photograph was wrongly rejected as “not a leaf”, invalid files produce clear errors, and database failure cannot affect prediction.

Weaknesses. The central weakness is model accuracy in the field. A top-1 accuracy of 48% means that, used naively, the tool would be wrong about as often as right on real photographs of supported crops; one in three field photographs produced a confidently presented wrong answer. The open-set mechanism, a key design contribution, rejected fewer than a third of untrained crops, and its feature-distance component did not separate known from unknown leaves on this benchmark. The project also lacks preserved held-out metrics from its own training run and has not been evaluated on the

Nepal-relevant crops (rice, banana, mango, citrus, cucumber) under field conditions, nor with real users. These weaknesses are serious, but they were found by the project's own evaluation, which is precisely the kind of independent measurement the literature review found to be missing.

The account decision. Making accounts compulsory was a trade-off, and it is worth evaluating honestly. In its favour, every saved check now has an owner who alone can read or delete it, and the same history appears on the phone and in the browser, which suits an extension worker who photographs leaves in the field and reviews them later. Against it, a farmer must now create an account before the first diagnosis, which adds friction and requires a working connection at sign-up; the earlier design had argued that no account should be needed to diagnose a leaf. The implementation limits the damage: the server keeps diagnosis open to any request, so the requirement could be relaxed in the clients without changing the API. The account features also add obligations the project has only partly met, notably password reset, account deletion and protection against password guessing (Table 28).

Was the approach right? In hindsight, the choice of backbone and training method was reasonable, but more effort was invested in model iterations on laboratory-dominated data than in field data. Barbedo (2018) and Singh *et al.* (2020) both indicate that variety of field data matters more than architecture; with only 10–64 field images per class, the model saw too little real-world variation. Equally, calibrating the open-set threshold on in-domain validation data meant it could not anticipate the shift in embedding distributions on field photographs. Calibrating on a held-out field set and evaluating rejection before deployment would have revealed this earlier.

Table 27: Comparison of CropSense AI with similar systems

Property	Mohanty *et al.* (2016)	PlantDoc (2020)	Cassava app (Ramcharan *et al.*, 2019)	Plantix (2024)	CropSense AI
Crops / classes	14 / 38	13 / 27	1 / 5	Many (undisclosed)	10 / 51 + Unknown
Independent field evaluation	Partial (31.4%)	Benchmark only	Yes	Not published	Yes (48.0% top-1)
Unknown-crop handling	No	No	No	Not published	Yes, measured (28.4%)
Uncertainty shown to user	No	No	Limited	Not published	Confidence bands, abstention
Visual explanation	No	No	No	No	Grad-CAM with caveat
Treatment guidance with safety data	No	No	No	Yes	Yes (PHI, hazard band, disclaimer)
Nepali language	No	No	No	Yes	Yes
Open, reproducible	Yes	Yes	Partly	No	Yes
Platforms	Research code	Dataset	Android (offline)	Android/iOS	Web, Android, REST API

Distinct contributions. Compared with the systems in Table 27, CropSense AI's novelty lies not in raw accuracy, where it is not competitive with a mature commercial product, but in the combination of properties and in its transparency. It is, to the author's knowledge, the only openly documented tool that (i) combines a transfer-learning

disease classifier with explicit, user-visible abstention and unknown-crop outcomes, (ii) qualifies its visual explanations, (iii) delivers structured, safety-conscious treatment guidance in Nepali, and (iv) publishes an honest evaluation of both field accuracy and untrained-crop rejection of the deployed system. Unlike academic prototypes it is usable by non-specialists today; unlike commercial tools its behaviour can be inspected, criticised and improved.

7.2 Impact

For farmers and extension services. The potential beneficiaries are smallholder farmers, agrovet staff and extension workers in Nepal. In its current state the tool is best positioned as a **triage and learning aid** used alongside human expertise: it can suggest likely candidates (the correct disease is in the top five for about four in five field photographs), explain typical symptoms and prevention in Nepali, and, importantly, remind users about label instructions, pre-harvest intervals and protective equipment. The selective-prediction results suggest a safer operating mode: when the model is at least 80% confident it is right about four times in five, so a deployment that shows a definitive answer only in that band and otherwise shows the top candidates with a referral to an agrovet would reduce harmful errors. The account history lets a user follow a problem through a season and show an agrovet exactly what was seen, at the price of an extra sign-up step and the storage of a username, an email address and small thumbnails.

Risks of impact. A wrong but confident answer could lead to an unnecessary or incorrect pesticide application, with economic, health and environmental costs. The disclaimers, confidence bands and cultural-first treatment ordering mitigate but do not eliminate this risk. Responsible use at scale would require the improvements set out in Section 8.2 and validation with agronomists.

For industry. The architecture, a single explainable inference service with cautious outputs shared by lightweight clients on free infrastructure, is a practical template for agritech start-ups and NGOs in low-resource settings, and the evaluation protocol (independent field benchmark, de-duplication against training data, system-level outcome reporting and untrained-crop testing) is directly reusable for procurement or quality assurance of such tools.

For academic discussion. The project adds a documented case to the evidence that laboratory accuracy dramatically overstates field performance, extends that evidence to open-set rejection by showing that a centroid-distance threshold calibrated in-domain failed to separate untrained crops under domain shift, and demonstrates in a deployed artefact that saliency explanations can be convincing for confident errors. It argues for reporting system-level outcomes, including abstentions and confidently wrong answers, rather than accuracy alone.

8. Conclusion

This project set out to address the lack of an accessible, trustworthy, Nepali-language means for smallholder farmers to identify crop leaf diseases, in a research landscape dominated by laboratory accuracies that do not transfer to the field and by models that neither explain themselves nor admit uncertainty. Following a design-science approach, it reviewed the literature, assembled and cleaned a 10-crop, 51-class dataset combining laboratory and field images, trained an EfficientNetB2 transfer-learning model with imbalance handling and field-realism augmentation, and built a complete system around it: an inference API with a leaf check, leaf cropping, a five-rule unknown gate and Grad-CAM explanations; a bilingual knowledge base with structured treatment guidance; user accounts with a hosted prediction history; and web and Android clients. The system was deployed publicly at zero cost and validated through automated tests, API edge-case tests and an independent field evaluation.

The engineering aim was achieved: CropSense AI works end to end and embodies responsible presentation of uncertain AI output. The evaluation, however, delivered the project's most important finding. On independent field photographs the model achieved 48.0% top-1 and 81.8% top-5 accuracy, compared with about 98.7% on laboratory validation data, and rejected only 28.4% of untrained crops, with the feature-distance rule failing to separate them. Confidence remained informative, with 81.8% accuracy for predictions made at 80% confidence or more. The value of the project therefore lies both in a usable, transparent tool and in clear evidence of what must change before such tools can be trusted in Nepali fields: more and better field data, domain-aware calibration and validation with real users.

8.1 Limitations

Table 28: Limitations, their impact and how they were addressed

Limitation	Impact on the project	How it was addressed
Small, web-sourced field dataset (10–64 images per class) without agronomist label verification	Limited field generalisation; residual label noise may bias training and the internal field score	Automated de-duplication, collision deletion and manual review; independent external benchmark for evaluation; documented as a priority for future work
Deployed run's held-out metrics (macro-F1, field-only score) not preserved	Internal performance of the deployed model cannot be reported	Reported only the preserved lab-run curves, clearly labelled; evaluated the deployed endpoint independently
External benchmark covers 17 of 51 classes and no rice, banana, mango, citrus or cucumber images; 226 images	Wide confidence intervals; no field evidence for key Nepali crops	De-duplicated against training data; reported intervals and per-class sample sizes; conclusions framed cautiously
Open-set threshold calibrated on in-domain validation data	Weak rejection of untrained crops under domain shift; higher false rejection on field images	Measured and reported explicitly; distinct user messages; softmax rules and Unknown class as additional layers
Single-label classification of one leaf	Cannot represent co-infection, severity, nutrient deficiency or pests beyond trained classes	Scope stated; photo guidance asks for one affected leaf; results framed as possible matches
No user study with farmers	Usability, comprehension of Nepali content and trust are untested	Accessibility guidelines and plain-language design followed; human studies excluded to remain within ethics and time constraints
Heuristic leaf pre-check and cropping (fixed colour thresholds)	May fail for unusual lighting or leaf colours	Safety rails return the original image; 0 field photographs falsely rejected in evaluation

Limitation	Impact on the project	How it was addressed
Free hosting (shared CPU, sleeping Space) and online-only inference	6.75 s latency; slow first request; unusable without connectivity	Efficient backbone; local Docker fallback; offline inference proposed
Input-size guard defect and development-grade security settings (open CORS, no rate limiting)	Potential memory exhaustion or abuse on public hosting	Found by testing and documented; no personal data stored, limiting exposure
Compulsory accounts without password reset, email verification, account deletion or login rate limiting	Friction for first-time users; a forgotten password loses the history; password guessing is not slowed; users cannot erase their own account	bcrypt hashes, 24-hour tokens, owner checks, TLS and secrets outside git; individual checks deletable; hardening listed in Section 8.2
Treatment content compiled from general sources, not locally registered product lists	Doses or products may not match Nepali registration	Cultural control listed first; label-and-agrovet disclaimer on every recommendation

8.2 Future Work

The following steps follow directly from the findings and limitations, in order of expected value:

1. **Collect expert-verified Nepali field data.** Partner with agricultural extension offices or universities to photograph at least 300 field images per class for Nepal's priority crops (rice, maize, potato, tomato, citrus and banana) across seasons, devices and regions, with labels confirmed by plant pathologists. This addresses the largest single weakness.
2. **Preserve and extend evaluation.** Re-run training with all notebook outputs saved and a fixed, versioned field test set; report macro-F1, per-class metrics, calibration error and a held-out untrained-crop rejection rate for every model, and keep PlantDoc as an external benchmark.
3. **Improve open-set rejection and calibration.** Calibrate thresholds on held-out field data; evaluate stronger methods such as Mahalanobis distance with shrinkage (Lee *et al.*, 2018), energy-based scores or an explicit leaf/crop verification model; apply temperature scaling (Guo *et al.*, 2017); and adopt a selective-prediction policy that shows a definitive answer only above a validated confidence level.
4. **Use the selected crop.** When the user specifies a crop, restrict or re-weight predictions to that crop's classes, which directly targets the observed tomato–potato and cross-crop confusions.
5. **Replace heuristics with learned localisation.** Train a lightweight leaf and lesion segmentation or detection model to replace the colour-threshold crop, and add robustness training for JPEG compression.
6. **Offline, on-device inference.** Convert the model to TensorFlow Lite with quantisation so the Android app can diagnose without connectivity, which is critical in rural areas, with the server retained for updates and explanations.
7. **Human-centred evaluation.** With ethical approval, run usability and comprehension studies with farmers and agrovet staff, including the Nepali text,

confidence bands and treatment cards, and measure whether the tool improves diagnostic decisions compared with current practice.

8. **Production hardening.** Fix the image-size guard, restrict CORS to known origins, add rate limiting and lockout on sign-in, password reset, email verification and self-service account deletion and export, add monitoring, and move to always-on hosting; extend the treatment knowledge base with locally registered products reviewed by experts, and add severity estimation and additional crops.

References

- Adadi, A. and Berrada, M. (2018) 'Peeking inside the black-box: a survey on explainable artificial intelligence (XAI)', *IEEE Access*, 6, pp. 52138–52160. doi:10.1109/ACCESS.2018.2870052.
- Adebayo, J., Gilmer, J., Muelly, M., Goodfellow, I., Hardt, M. and Kim, B. (2018) 'Sanity checks for saliency maps', in *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)*. Red Hook, NY: Curran Associates, pp. 9505–9515.
- Barbedo, J.G.A. (2018) 'Impact of dataset size and variety on the effectiveness of deep learning and transfer learning for plant disease classification', *Computers and Electronics in Agriculture*, 153, pp. 46–53. doi:10.1016/j.compag.2018.08.013.
- Barbedo, J.G.A. (2019) 'Plant disease identification from individual lesions and spots using deep learning', *Biosystems Engineering*, 180, pp. 96–107. doi:10.1016/j.biosystemseng.2019.02.002.
- BCS (2022) *BCS Code of Conduct*. Swindon: BCS, The Chartered Institute for IT. Available at: <https://www.bcs.org/membership-and-registrations/become-a-member/bcs-code-of-conduct/> (Accessed: 15 September 2026).
- Beck, K. *et al.* (2001) *Manifesto for Agile Software Development*. Available at: <https://agilemanifesto.org> (Accessed: 15 September 2026).
- Buda, M., Maki, A. and Mazurowski, M.A. (2018) 'A systematic study of the class imbalance problem in convolutional neural networks', *Neural Networks*, 106, pp. 249–259. doi:10.1016/j.neunet.2018.07.011.
- Data Protection Act 2018*, c. 12. London: The Stationery Office. Available at: <https://www.legislation.gov.uk/ukpga/2018/12/contents> (Accessed: 15 September 2026).
- Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K. and Fei-Fei, L. (2009) 'ImageNet: a large-scale hierarchical image database', in *2009 IEEE Conference on Computer Vision and Pattern Recognition*. Miami, FL: IEEE, pp. 248–255. doi:10.1109/CVPR.2009.5206848.
- Ferentinos, K.P. (2018) 'Deep learning models for plant disease detection and diagnosis', *Computers and Electronics in Agriculture*, 145, pp. 311–318. doi:10.1016/j.compag.2018.01.009.
- Geifman, Y. and El-Yaniv, R. (2017) 'Selective classification for deep neural networks', in *Advances in Neural Information Processing Systems 30 (NIPS 2017)*. Red Hook, NY: Curran Associates, pp. 4878–4887.
- Geng, C., Huang, S.-J. and Chen, S. (2021) 'Recent advances in open set recognition: a survey', *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43(10), pp. 3614–3631. doi:10.1109/TPAMI.2020.2981604.
- Guo, C., Pleiss, G., Sun, Y. and Weinberger, K.Q. (2017) 'On calibration of modern neural networks', in *Proceedings of the 34th International Conference on Machine Learning*. PMLR 70, pp. 1321–1330.
- He, K., Zhang, X., Ren, S. and Sun, J. (2016) 'Deep residual learning for image recognition', in *2016 IEEE Conference on Computer Vision and Pattern Recognition*. Las Vegas, NV: IEEE, pp. 770–778. doi:10.1109/CVPR.2016.90.

- Hendrycks, D. and Gimpel, K. (2017) 'A baseline for detecting misclassified and out-of-distribution examples in neural networks', in *5th International Conference on Learning Representations (ICLR 2017)*. Toulon, France.
- Hevner, A.R., March, S.T., Park, J. and Ram, S. (2004) 'Design science in information systems research', *MIS Quarterly*, 28(1), pp. 75–105. doi:10.2307/25148625.
- Hughes, D.P. and Salathé, M. (2015) 'An open access repository of images on plant health to enable the development of mobile disease diagnostics', *arXiv preprint*, arXiv:1511.08060.
- Johnson, J.M. and Khoshgoftaar, T.M. (2019) 'Survey on deep learning with class imbalance', *Journal of Big Data*, 6, article 27. doi:10.1186/s40537-019-0192-5.
- Jones, M., Bradley, J. and Sakimura, N. (2015) *JSON Web Token (JWT)*. RFC 7519. Internet Engineering Task Force. doi:10.17487/RFC7519.
- Kamilaris, A. and Prenafeta-Boldú, F.X. (2018) 'Deep learning in agriculture: a survey', *Computers and Electronics in Agriculture*, 147, pp. 70–90. doi:10.1016/j.compag.2018.02.016.
- Keras (2026) *Keras Applications*. Available at: <https://keras.io/api/applications/> (Accessed: 15 September 2026).
- Kingma, D.P. and Ba, J. (2015) 'Adam: a method for stochastic optimization', in *3rd International Conference on Learning Representations (ICLR 2015)*. San Diego, CA.
- Koh, P.W. et al. (2021) 'WILDS: a benchmark of in-the-wild distribution shifts', in *Proceedings of the 38th International Conference on Machine Learning*. PMLR 139, pp. 5637–5664.
- Lee, K., Lee, K., Lee, H. and Shin, J. (2018) 'A simple unified framework for detecting out-of-distribution samples and adversarial attacks', in *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)*. Red Hook, NY: Curran Associates, pp. 7167–7177.
- Liu, J. and Wang, X. (2021) 'Plant diseases and pests detection based on deep learning: a review', *Plant Methods*, 17, article 22. doi:10.1186/s13007-021-00722-9.
- Loshchilov, I. and Hutter, F. (2017) 'SGDR: stochastic gradient descent with warm restarts', in *5th International Conference on Learning Representations (ICLR 2017)*. Toulon, France.
- Micikevicius, P. et al. (2018) 'Mixed precision training', in *6th International Conference on Learning Representations (ICLR 2018)*. Vancouver, Canada.
- MoALD (2023) *Statistical Information on Nepalese Agriculture 2078/79 (2021/22)*. Kathmandu: Ministry of Agriculture and Livestock Development, Government of Nepal.
- Mohanty, S.P., Hughes, D.P. and Salathé, M. (2016) 'Using deep learning for image-based plant disease detection', *Frontiers in Plant Science*, 7, article 1419. doi:10.3389/fpls.2016.01419.
- Müller, R., Kornblith, S. and Hinton, G.E. (2019) 'When does label smoothing help?', in *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*. Red Hook, NY: Curran Associates, pp. 4694–4703.

- Northcutt, C.G., Athalye, A. and Mueller, J. (2021) 'Pervasive label errors in test sets destabilize machine learning benchmarks', in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1 (NeurIPS 2021)*.
- Payne, A. (2018) *Farming in Nepal* [photograph]. Wikimedia Commons, licensed under CC BY-SA 4.0. Available at: https://commons.wikimedia.org/wiki/File:Farming_in_Nepal.jpg (Accessed: 14 September 2026).
- Picon, A., Alvarez-Gila, A., Seitz, M., Ortiz-Barredo, A., Echazarra, J. and Johannes, A. (2019) 'Deep convolutional neural networks for mobile capture device-based crop disease classification in the wild', *Computers and Electronics in Agriculture*, 161, pp. 280–290. doi:10.1016/j.compag.2018.04.002.
- Plantix (2024) *Plantix: your crop doctor*. PEAT GmbH. Available at: <https://plantix.net> (Accessed: 15 September 2026).
- Provos, N. and Mazières, D. (1999) 'A future-adaptable password scheme', in *Proceedings of the 1999 USENIX Annual Technical Conference, FREENIX Track*. Monterey, CA: USENIX Association, pp. 81–91.
- Ramcharan, A., Baranowski, K., McCloskey, P., Ahmed, B., Legg, J. and Hughes, D.P. (2017) 'Deep learning for image-based cassava disease detection', *Frontiers in Plant Science*, 8, article 1852. doi:10.3389/fpls.2017.01852.
- Ramcharan, A., McCloskey, P., Baranowski, K., Mbilinyi, N., Mrisho, L., Ndalawwa, M., Legg, J. and Hughes, D.P. (2019) 'A mobile-based deep learning model for cassava disease diagnosis', *Frontiers in Plant Science*, 10, article 272. doi:10.3389/fpls.2019.00272.
- Sandler, M., Howard, A., Zhu, M., Zhmoginov, A. and Chen, L.-C. (2018) 'MobileNetV2: inverted residuals and linear bottlenecks', in *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*. Salt Lake City, UT: IEEE, pp. 4510–4520. doi:10.1109/CVPR.2018.00474.
- Savary, S., Willocquet, L., Pethybridge, S.J., Esker, P., McRoberts, N. and Nelson, A. (2019) 'The global burden of pathogens and pests on major food crops', *Nature Ecology & Evolution*, 3(3), pp. 430–439. doi:10.1038/s41559-018-0793-y.
- Selvaraju, R.R., Cogswell, M., Das, A., Vedantam, R., Parikh, D. and Batra, D. (2017) 'Grad-CAM: visual explanations from deep networks via gradient-based localization', in *2017 IEEE International Conference on Computer Vision*. Venice, Italy: IEEE, pp. 618–626. doi:10.1109/ICCV.2017.74.
- Shorten, C. and Khoshgoftaar, T.M. (2019) 'A survey on image data augmentation for deep learning', *Journal of Big Data*, 6, article 60. doi:10.1186/s40537-019-0197-0.
- Singh, D., Jain, N., Jain, P., Kayal, P., Kumawat, S. and Batra, N. (2020) 'PlantDoc: a dataset for visual plant disease detection', in *Proceedings of the 7th ACM IKDD CoDS and 25th COMAD*. Hyderabad, India: ACM, pp. 249–253. doi:10.1145/3371158.3371196.
- Sokolova, M. and Lapalme, G. (2009) 'A systematic analysis of performance measures for classification tasks', *Information Processing & Management*, 45(4), pp. 427–437. doi:10.1016/j.ipm.2009.03.002.
- Strange, R.N. and Scott, P.R. (2005) 'Plant disease: a threat to global food security', *Annual Review of Phytopathology*, 43, pp. 83–116. doi:10.1146/annurev.phyto.43.113004.133839.

- Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J. and Wojna, Z. (2016) 'Rethinking the Inception architecture for computer vision', in *2016 IEEE Conference on Computer Vision and Pattern Recognition*. Las Vegas, NV: IEEE, pp. 2818–2826. doi:10.1109/CVPR.2016.308.
- Tan, M. and Le, Q.V. (2019) 'EfficientNet: rethinking model scaling for convolutional neural networks', in *Proceedings of the 36th International Conference on Machine Learning*. PMLR 97, pp. 6105–6114.
- Too, E.C., Yujian, L., Njuki, S. and Yingchun, L. (2019) 'A comparative study of fine-tuning deep learning models for plant disease identification', *Computers and Electronics in Agriculture*, 161, pp. 272–279. doi:10.1016/j.compag.2018.03.032.
- W3C (2023) *Web Content Accessibility Guidelines (WCAG) 2.2*. W3C Recommendation, 5 October 2023. Available at: <https://www.w3.org/TR/WCAG22/> (Accessed: 15 September 2026).
- Wirth, R. and Hipp, J. (2000) 'CRISP-DM: towards a standard process model for data mining', in *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*. Manchester, pp. 29–39.
- World Bank (2024) *World Development Indicators: Nepal (agriculture, forestry and fishing value added, % of GDP; employment in agriculture, % of total employment)*. Washington, DC: The World Bank. Available at: <https://data.worldbank.org/country/nepal> (Accessed: 15 September 2026).
- Zauner, C. (2010) *Implementation and benchmarking of perceptual image hash functions*. Master's thesis. Upper Austria University of Applied Sciences, Hagenberg.
- Zhuang, F., Qi, Z., Duan, K., Xi, D., Zhu, Y., Zhu, H., Xiong, H. and He, Q. (2021) 'A comprehensive survey on transfer learning', *Proceedings of the IEEE*, 109(1), pp. 43–76. doi:10.1109/JPROC.2020.3004555.

Appendices

The appendices provide supporting material that is too detailed for the main report.

A. Project Timeline

Figure 32 shows the project schedule. Implementation dates are taken from the Git commit history and model file timestamps (repository created 28 June 2026; weights tracked with Git LFS 31 July; field images added 26 August; deployed model trained 6 September; public deployment 13 September; accounts and hosted database 23 September; APK rebuilt 24 September). Planning and literature review dates preceding the repository are approximate.

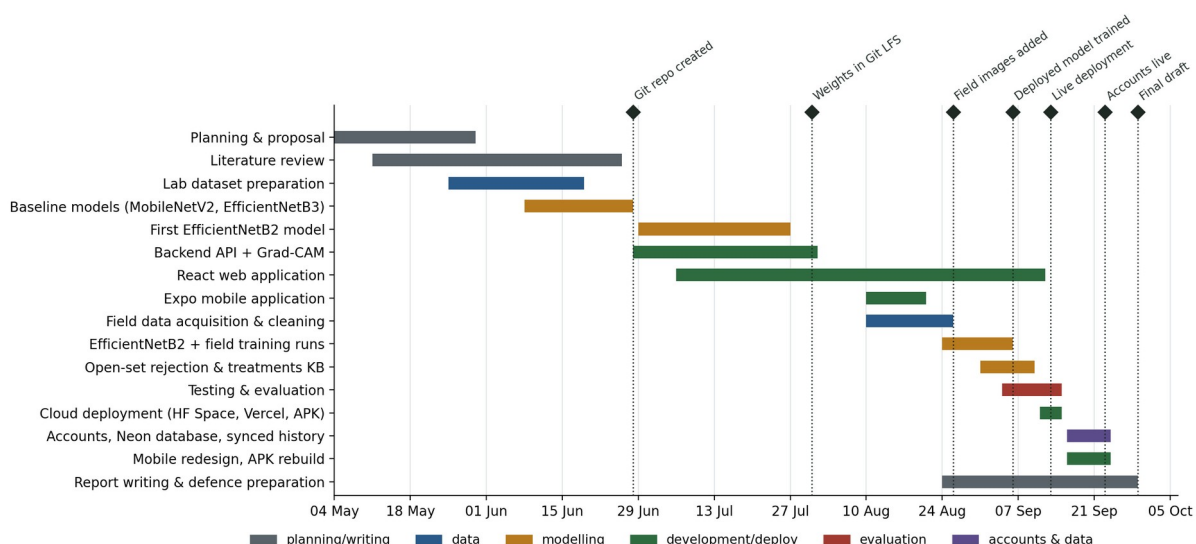


Figure 32: Project Gantt chart with milestones

B. Feasibility, Risks, and Ethical Aspects

B.1. Revised Feasibility

Time. The project was delivered within the planned window, although model iteration and field data collection took longer than planned and compressed evaluation into the final fortnight; this is why training outputs were not re-generated and an external benchmark was used instead. **Data.** Public laboratory image collections and web-sourced field images provided sufficient data to build a working model, but not enough field data for high field accuracy. **Hardware and software.** All training used free Google Colab T4 GPUs, with resume-safe checkpointing to survive disconnections; development used a standard laptop and open-source software; deployment used free tiers of Hugging Face, Vercel and Expo. Disk space on the development machine was a practical constraint, handled by sparse downloads and cloud builds. **Skills.** The project required skills across deep learning, backend and frontend development, mobile development and cloud deployment; gaps (for example Keras 3 changes and Hugging Face deployment requirements) were closed through documentation and systematic

debugging. The completed, deployed artefact demonstrates that the project was feasible within its constraints; the principal constraint on quality was access to expert-labelled field data rather than technology.

B.2. Updated Risk Analysis and Mitigation

Table 29: Risk assessment and mitigation

Potential risk	Likelihood	Impact	Mitigation strategy (and outcome)
Insufficient or unrepresentative field data leads to poor real-world accuracy	High	High	Merge field images, field augmentation, leaf cropping, independent evaluation. <i>Outcome: risk materialised (48% top-1); documented; expert data collection proposed.</i>
Free GPU sessions disconnect or quotas run out during training	High	Medium	Per-epoch checkpoints to Google Drive with automatic resume; pre-resized images to shorten epochs. <i>Outcome: runs completed.</i>
Free hosting cannot run TensorFlow (memory) or requires payment	Medium	High	Parameter-efficient backbone; compare providers; Docker fallback. <i>Outcome: Docker Spaces became paid; migrated to Gradio SDK.</i>
Dependency or framework changes silently break features (e.g. Grad-CAM)	Medium	Medium	Exact version pins; start-up logging; tests. <i>Outcome: Keras 3 break found and fixed.</i>
Users act on a wrong diagnosis and misuse pesticides	Medium	High	Possible-match wording, confidence bands, abstention, label-and-agrovet disclaimer, cultural control first. <i>Outcome: residual risk remains; selective prediction proposed.</i>
Loss of code, model or data	Low	High	Git and GitHub; Git LFS on Hugging Face; Drive checkpoints; local copies. <i>Outcome: no loss.</i>
Leak or breach of the accounts database	Low	High	Passwords stored only as bcrypt hashes; no full-size photos; TLS; secrets outside git; owner checks. <i>Outcome: no incident; rate limiting, account deletion and a privacy notice proposed.</i>
Copyright or licence issues with third-party images	Medium	Medium	Field images used only for training, not redistributed in the public repository; licensed hero image credited. <i>Outcome: provenance documentation still to be completed.</i>
Live service unavailable during assessment or demonstration	Medium	Medium	Wake the Space before use; local Docker Compose stack and installed APK as fallbacks.

B.3 Revised Ethical Considerations

Human participants and personal data. The project involved no human participants, interviews or surveys, and therefore did not require informed consent procedures. Since 23 September 2026 the deployed system requires an account, so it now processes limited personal data: a username, an email address and a bcrypt hash of the password, and, for each check, the result, a 160×120 thumbnail and the Grad-CAM overlay, which contains a blended copy of the photograph. Full-size photographs are processed in memory and discarded. The data are held in a Neon database in Singapore, reached over TLS, and each record can be read or deleted only by its owner. This design follows the data-minimisation principle of the Data Protection Act 2018, but it does not yet meet every obligation of a public service: there is no privacy notice on the sign-up screen, no self-service account deletion or data export, no defined retention period, and the data are stored outside both Nepal and the UK. These must be addressed, and the choice of region reviewed, before any use with real farmers. Passwords are hashed with bcrypt and tokens are signed with a secret held outside the code.

Datasets and intellectual property. Laboratory images come from publicly available research collections, and PlantDoc is published for research use (Singh *et al.*, 2020). Field images were collected from the web for non-commercial research and model training only; they are not redistributed in the public code repository, and their individual licences have not been verified, which would need to be resolved, for example by replacing them with consented or openly licensed photographs, before any commercial use. The home-page photograph is used under CC BY-SA 4.0 with attribution.

Social impact and responsible AI. The main ethical risk is harm from over-reliance on incorrect predictions, particularly unnecessary or incorrect pesticide use affecting farmer income, health and the environment. The evaluation shows this risk is real. The system therefore frames results as possible matches, communicates uncertainty and abstains, qualifies explanations, lists non-chemical measures first, and instructs users to read product labels, observe pre-harvest intervals, wear protective equipment and confirm advice with an agrovet or agriculture office. The limitations are reported transparently rather than hidden, consistent with the BCS Code of Conduct's requirements to act in the public interest and within one's professional competence (BCS, 2022). Treatment content is general guidance and must not be presented as certified agronomic advice. Bias is also a concern: the model performs unevenly across classes and has not been validated on Nepal's priority crops in the field, so any real deployment should be preceded by local validation and expert review.

C. Additional Screenshots

स्याउ

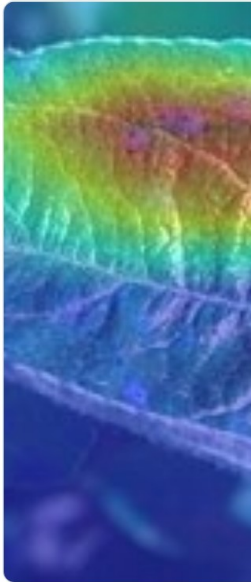
सम्भावित रोग

स्याउको सिडार सिन्दुरे रोग

विश्वसनीयता

सिडार सिन्दुरे रोग *Gymnosporangium juniperi-virginianae* नामक दुसीबाट लाग्छ। यो रोग फैलिन स्याउ र सिडार/जुनिपर चम्किलो सुन्तला-पहेँलो थोप्ला बनाउँछ र फल बिग्रन्छ।

दृश्य व्याख्या

तपाईंको तस्बिर

मो

रङ्गिएका भागहरूले तस्बिरका कुन भागले मोडेललाई सबैभन्दा बढी प्रभाव पारे भन्ने देखाउँछ। यो दृश्य व्याख्याले रोगको पुष्टि गर्दैन।

Figure 33: Result page in Nepali: every content field, including the confidence band and explanation text, is translated

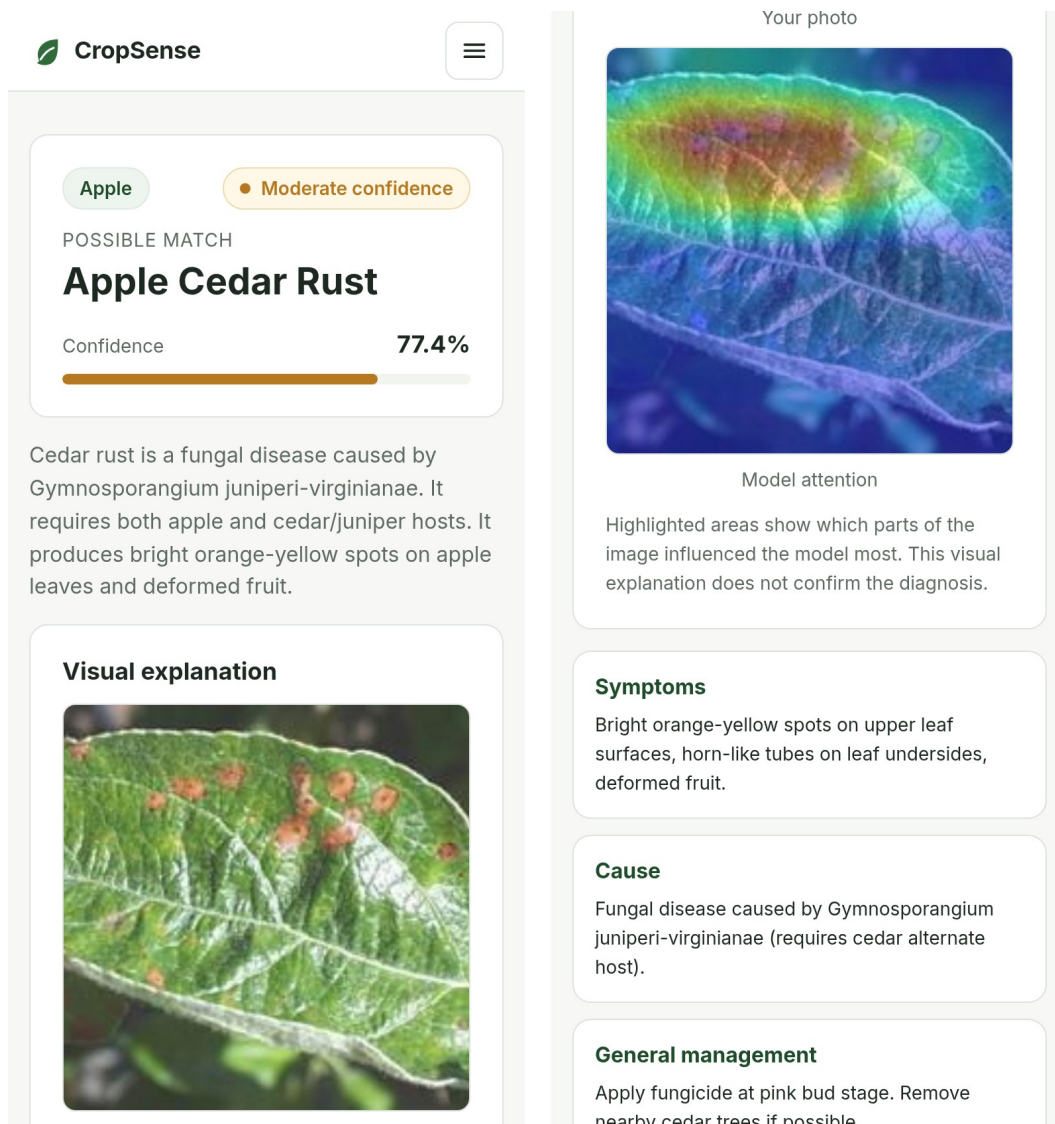


Figure 34: Responsive web interface at phone width (390 px): result and treatment sections

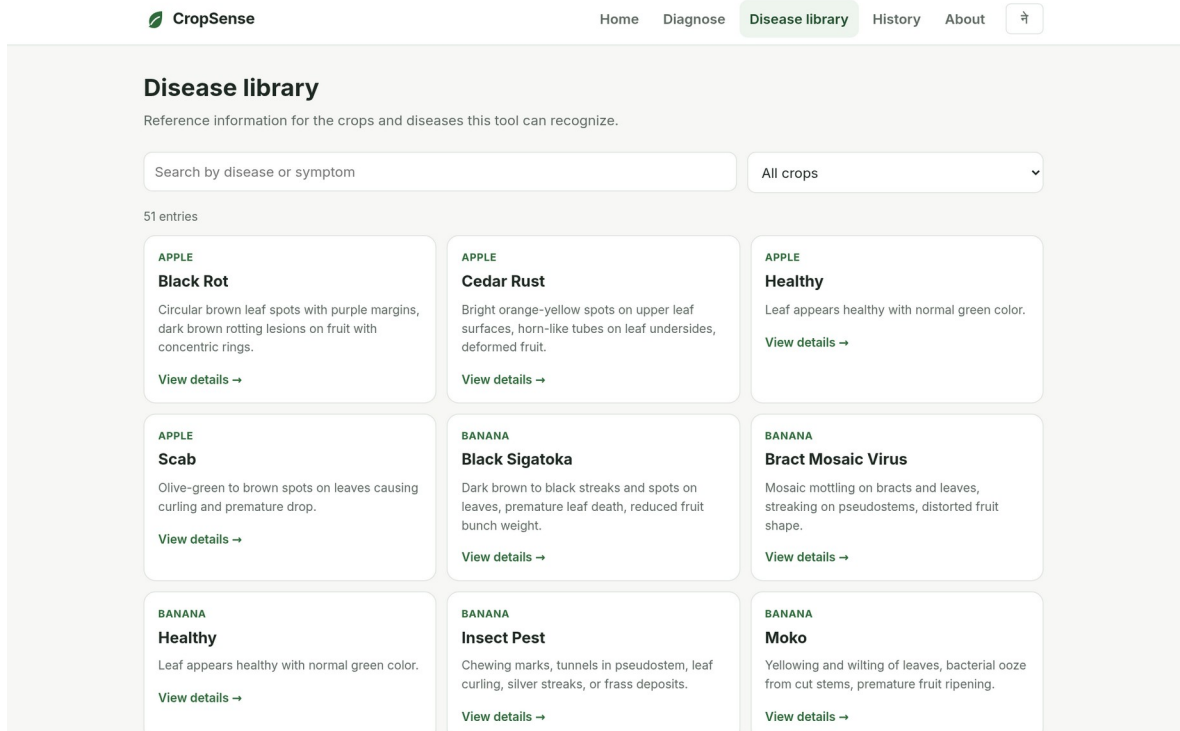


Figure 35: Disease library listing all supported conditions

Android application. The native Android client shares the same API and bilingual result content. Screenshots from a physical device should be inserted below.

[Insert screenshots of the installed Android APK: home screen, crop selection, result with Grad-CAM, and Nepali view]

Figure 36: Android application screenshots (to be inserted from a physical device)

D. Example API Response (abridged)

```
POST https://bikkii-cropsense.hf.space/predict
multipart form: file=leaf.jpg, crop_type=Apple
{
  "disease": "Apple Cedar Rust",
  "disease_np": "स्याउको सिडार सिन्दुरे रोग",
  "raw_class": "Apple_Cedar_Rust", "crop_type": "Apple",
  "confidence": 85.4, "low_confidence": false,
  "entropy": 0.2341, "ood_score": 0.7851,
  "is_unknown": false, "not_leaf": false, "unknown_reason": null, "crop_mismatch": false,
  "top_5_predictions": [ {"disease": "Apple Cedar Rust", "confidence": 85.4},
    {"disease": "Apple Black Rot", "confidence": 2.03}, ... ],
  "symptoms": "Bright orange-yellow spots on upper leaf surfaces, ...",
}
```

```

"treatments": [
  {"kind": "cultural", "active": "Remove nearby juniper/cedar trees ...", "band": "none"},
  {"kind": "fungicide", "active": "Mancozeb 75% WP", "dose": "2 g per litre of water",
    "phi_days": 14, "interval_days": 10, "band": "blue"},
  {"kind": "fungicide", "active": "Myclobutanil 10% WP", "dose": "1 g per litre of water",
    "phi_days": 21, "interval_days": 14, "band": "yellow"} ],
"treatment_disclaimer": "Always read and follow the product label. ...",
"gradcam_image": "data:image/jpeg;base64,/9j/4AAQSkZJRg..."
}

```

Values are taken from an actual response of the deployed model for the PlantDoc apple-rust image shown in Figure 17; long text fields and the base64 heatmap are truncated.

E. Full Per-Class PlantDoc Results

Table 30: Per-folder PlantDoc results for the deployed model

PlantDoc folder	CropSense class	n	Top-1 correct	Top-5 correct	Shown & correct	Declined
Apple leaf	Apple Healthy	9	6	7	6	2
Apple rust leaf	Apple Cedar Rust	8	5	6	5	2
Apple Scab Leaf	Apple Scab	10	4	7	3	3
Corn leaf blight	Maize Northern Leaf Blight	12	8	12	8	1
Corn rust leaf	Maize Common rust	9	8	8	8	1
grape leaf	Grape Healthy	12	10	12	10	1
grape leaf black rot	Grape Black Rot	8	2	6	1	5
Potato leaf early blight	Potato Early Blight	7	3	6	2	2
Potato leaf late blight	Potato Late Blight	8	5	8	4	1
Tomato Early blight leaf	Tomato Early blight	9	6	9	5	3
Tomato leaf	Tomato Healthy	8	1	6	1	3
Tomato leaf bacterial spot	Tomato Bacterial Spot	8	0	4	0	3
Tomato leaf late blight	Tomato Late blight	9	1	5	1	2
Tomato leaf mosaic virus	Tomato Mosaic Virus	9	2	5	2	1
Tomato leaf yellow virus	Tomato Yellow Leaf Curl Virus	5	3	4	3	0
Tomato mold leaf	Tomato Leaf Mold	6	2	5	2	2
Tomato Septoria leaf spot	Tomato Septoria Leaf Spot	11	5	11	3	2
Corn Gray leaf spot (Unsupported disease)	—	4	—	—	—	1
Bell_pepper leaf (Untrained crop)	—	8	—	—	—	3
Bell_pepper leaf spot (Untrained crop)	—	9	—	—	—	3
Blueberry leaf (Untrained crop)	—	11	—	—	—	7
Cherry leaf (Untrained crop)	—	10	—	—	—	1
Peach leaf (Untrained crop)	—	7	—	—	—	1
Raspberry leaf (Untrained crop)	—	7	—	—	—	4
Soyabean leaf (Untrained crop)	—	8	—	—	—	2
Squash Powdery mildew leaf (Untrained crop)	—	6	—	—	—	0
Strawberry leaf (Untrained crop)	—	8	—	—	—	0

F. Repository Structure and Running the Artefact

```

CropSense/
├── backend/           FastAPI service (main.py), auth.py, db.py, disease_info.json,
                        treatments.json; last_final_model/ = deployed weights,
                        class_names.json and ood_stats.npz
├── frontend/         React 19 + Vite web client (src/pages, src/components, src/lib)
├── mobile/           Expo SDK 57 Android client (App.js, AuthScreen.js, HistoryScreen.js,
                        ProfileScreen.js, Sidebar.js, ResultView.js, api.js, auth.js)

```

```
├── scripts/           Colab training notebooks; field image download/cleaning scripts
├── database/         schema.sql: users and predictions tables (mirrors init_db)
├── docker-compose.yml db, backend, frontend (nginx), adminer
└── DEPLOYMENT.md     deployment guide (Hugging Face Space, Vercel, EAS)
```

```
# Run the full stack locally
```

```
git lfs pull && docker compose up --build
```

```
# web: http://localhost:3000    API docs: http://localhost:8000/docs
```

```
# Run the web client against the live API
```

```
cd frontend && npm install
```

```
VITE_API_BASE_URL=https://bikkii-cropsense.hf.space npm run dev
```

```
# Run the automated tests
```

```
cd frontend && npx vitest run
```

```
# Build the Android APK
```

```
cd mobile && npx eas-cli build -p android --profile preview
```